Team Number :	apmcm24207440
Problem Chosen :	C

2024 APMCM summary sheet

Abstract

According to the 2021 China Pet Industry White Paper, the population of pet dogs and cats in urban China reached 68.44 million. As the number of pet owners increases, the production and import-export scale of pet food in China have been continuously expanding. According to statistics from the China Feed Industry Association, the national pet feed production in 2021 reached 1.13 million tons, with 357 pet feed production companies across the country. Based on data from China Customs, the import-export scale of dog and cat food in China increased from $740 million in 2013 to $1.916 billion in 2022, with the total trade volume rising from 123,000 tons to 375,000 tons.

The pet industry has developed rapidly in recent years, becoming an important part of the global economy, particularly in China, where the number of pets and related industries have grown significantly. However, changes in international economic policies, such as tariff adjustments, have introduced uncertainties to China’s pet food exports. This paper uses data from 2019 to 2023 and employs time series analysis, regression models, and Logistic growth models to systematically analyze the development trends of the pet industry in China and globally, while simulating the potential impact of international policies on China’s pet food exports. The study finds that the number of pet cats in China is growing rapidly, with significant production potential in pet food. However, exports are heavily influenced by international policies. This paper proposes strategies such as diversifying market expansion, developing high-end products, and responding to policies, offering recommendations for the sustainable development of China’s pet industry.

Keywords: Pet Industry, China, Pet Food Demand, Economic Development, Import-Export, Sustainable Developmen

Contents

2024 APMCM summary sheet 1
Abstract 1
Ⅰ.Introduction 2
1.1 Background 2
1.2 Research Objectives 2
1.3 Significance of the Research 2
Ⅱ. The Description of the Problem 2
2.1Restatement of the Problem: 2
2.2 Problem Analysis 3
2.2.1 Problem 1: 3
2.2.2 Problem 2: 4
2.2.3 Problem 3: 4
Ⅲ.Models 4
3.1 Model Introduction 4
3.2Assumptions 5
3.3Solution and Result 7
3.3.1Question one： 7
3.3.2 Question two： 10
3.3.3Question three： 12
3.3.4 Question 4： 15
3.4Analysis of the Result 18
Ⅴ.Future Work 22
Pet industry development characteristics 22
1. Economic policy 22
2. Technology and Digital Transformation Policy 23
3.Social policy 23
4. International trade and external policy 23
Summary 24
Ⅵ. References 24
Ⅶ.Appendix 25

Ⅰ.Introduction

1.1 Background

With the development of the global economy and the acceleration of urbanization, pets have become an important part of many households. Relevant data shows that the number of pets worldwide has been increasing year by year, and industries related to pet food, pet healthcare, and pet products have flourished, driving the rapid rise of the global pet economy. In particular, in China, although the pet industry started later, it has developed at an astonishing pace and has become an important part of the global pet market.

Between 2019 and 2023, the number of pet cats and dogs in China grew rapidly, with the total production value and export value of pet food also showing high growth trends. However, with the increasing complexity of the international economic environment, particularly the adjustment of tariff policies in European and American countries, China's pet food exports face new challenges, which urgently require in-depth analysis of the current development trends and the formulation of corresponding strategies.

1.2 Research Objectives

To analyze the development of China's pet industry over the past five years and predict the number of pets and market size for the next three years;

To analyze the global pet industry and predict the demand for pet food in the next three years;

To forecast the growth potential of China's pet food production and exports;

To simulate the impact of international policies on China's pet food exports and propose countermeasures.

1.3 Significance of the Research

By analyzing the historical data of the pet industry in China and globally, this research aims to predict the development trends of the pet industry over the next three years, assess the impact of changes in international economic policies on China's pet food exports, and propose optimization strategies. This research is of great significance for promoting the sustainable development of China's pet industry and provides scientific basis for businesses to respond to changes in the international market.

Ⅱ. The Description of the Problem

2.1Restatement of the Problem:

In recent years, with the continuous improvement of economic levels and changing consumer attitudes, the global pet industry has experienced rapid growth, particularly in China. The number of pets and the development of related industries have shown a strong upward trend. This phenomenon has driven the rapid development of sectors such as pet food, pet products, and pet healthcare, making the pet industry an increasingly important force in the global economy.

In China, the growth of the pet industry has been particularly prominent. From pet food to pet healthcare, from pet products to smart devices, the market's diversification has provided vast business opportunities for related industries. On a global scale, China has

become one of the largest and fastest-growing pet food exporters. According to customs data, China's annual pet food exports have grown by more than 10%, with major export markets including the United States, Europe, and Southeast Asia.

Additionally, international standards for pet food quality, testing regulations, and safety certifications have become increasingly stringent. These external policy uncertainties have placed higher operational costs and compliance pressures on Chinese pet food exporters in the international market.

This article, based on data from 2019 to 2023, uses time series analysis, regression models, and logistic growth models to analyze the current situation and future trends of China's pet industry. Time series analysis helps identify the cyclical fluctuations in the pet food market and reveals a generally stable growth trend in the Chinese pet industry. Regression models are used to analyze the relationship between multiple factors and the demand for pet food, with results showing that income levels, pet numbers, and consumption upgrades significantly influence market demand. The logistic growth model is used to simulate the growth potential of China's pet food market and predict the market's growth trend in the coming years.

The article proposes several key strategies and policy recommendations to promote the sustainable development of China's pet industry:

1. To mitigate the risks posed by changes in trade policies, companies should actively explore new export markets. Strengthening trade cooperation with emerging markets will help diversify risks and increase market share globally.

2. To address trade barriers in international markets, the government can engage in diplomatic efforts to reduce tariffs and simplify inspection and quarantine procedures, thereby creating a more favorable external environment for Chinese pet food companies.

3. The government can also encourage companies to enhance innovation and improve product quality to meet increasingly strict quality standards in international markets.

4. With the development of global e-commerce platforms, cross-border e-commerce has become an important channel for Chinese pet food exports. Companies can use platforms such as Alibaba International, Amazon, and eBay to promote Chinese-made high-quality pet food worldwide. Additionally, through digital marketing and social media promotion, companies can directly engage with global consumers, increasing brand visibility and market share.

2.2 Problem Analysis

2.2.1 Problem 1:

Analyze the development of China's pet industry and predict its growth in the next three years

Data Analysis: From 2019 to 2023, the number of cats steadily increased (from 44.12 million to 69.80 million), while the number of dogs fluctuated but remained relatively stable (5.175 million in 2023, 5.222 million in 2020). This indicates a clear increase in demand for cats in China's pet market, which may be related to trends in pet companionship and changes in lifestyle.

Analysis Factors: The growth of the pet industry may be related to several factors, such as increased per capita income, urbanization, and changes in social structure (e.g., more single-person households, the trend of young people owning pets).

2.2.2 Problem 2:

Analyze the development of the global pet industry and predict the demand for pet food over the next three years

Data Analysis: The number of pets in the United States is relatively large but stable (7.38 million cats and 8.01 million dogs in 2023). While the overall number has not fluctuated significantly, demand in the pet market remains strong. In European countries (France, Germany), pet numbers show different trends, but they generally remain stable, with fewer dogs in France and Germany, which may reflect differing pet-raising habits across countries.

From this comparison, we can see that pet industries in developed countries such as the U.S. and Europe also show growth or relative stability.

2.2.3 Problem 3:

Analyze the current situation of China's pet food industry and predict the output and export volume over the next three years.

Data Analysis: The value of China's pet food output in 2023 shows that the pet food industry has experienced growth in recent years. Regarding exports, while there was a sharp decline after 2019, it has gradually recovered.

Development Trend: Due to the growing global market size, China's pet food industry is likely to continue its rapid growth.

2.2.4 Problem 4: Evaluate the impact of economic policies in European and American countries on China's pet food industry and propose development strategies

Data Analysis: Economic policies in Europe and the U.S. (such as tariff policies) can directly affect the export of Chinese pet food. Increased tariffs may reduce the competitiveness of Chinese pet food, leading to a decline in export volume. Additionally, domestic pet food brands in Europe and the U.S. may benefit from changes in policy, increasing competition pressure on Chinese exports.

Ⅲ.Models

3.1 Model Introduction

3.1.1 Decision Tree Regression Model:

The decision tree regression model is a regression method based on decision trees, used to solve regression problems. The goal of decision tree regression is to partition the

data into different regions based on features, and within each region, make predictions using simple models (such as constant values or averages).

Mean Squared Error (MSE): MSE is the most commonly used performance metric for regression, measuring the squared error between predicted values and actual values.

Formula:

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

3.1.2 Random Forest Regression Model:

The random forest regression model solves regression problems by constructing multiple decision trees, thereby improving the accuracy and robustness of predictions. Compared to a single decision tree, random forest reduces the model's variance by aggregating the predictions of multiple decision trees, helping to avoid overfitting and achieve stronger generalization ability.

The final output of the random forest regression is the average of the predicted values from all the trees. That is, for each input sample x, the predicted value from the random forest regression is the arithmetic mean of the predicted values from all the trees:

$$\hat{y} = \frac{1}{N}\sum_{i=1}^{N} T_i(x)$$

3.1.4 Exponential Smoothing

- Simple Exponential Smoothing: Suitable for stationary time series. The basic idea is to compute a weighted average of historical data, with more recent data receiving higher weights.

- Holt’s Linear Trend Method: Suitable for time series with a trend. It takes into account both the level and the trend of the data.

- Holt-Winters Seasonal Method: Suitable for time series with seasonality. This model accounts for the level, trend, and seasonal components of the data.

Application Scenarios: Exponential smoothing methods are useful for time series data that are stationary, have a trend, or exhibit seasonality.

3.2Assumptions

In this analysis, we assume that all the involved data (such as the number of pet cats and dogs, demand for pet food, and the pet food import/export rate) is accurate and reliable. Specifically, we make the following assumptions:

3.2.2. Demand for Pet Food

Assumption on Data Accuracy: We assume that demand data for pet food comes from large retailers, e-commerce platforms, pet food manufacturers' sales data, and regular reports from industry associations. We further assume that these data reflect actual market consumption trends, accounting for variations in demand across different types of pet food (e.g., cat food, dog food, treats) and different price ranges and packaging formats (e.g., dry food, wet food).

Margin of Error: We assume that the demand data may experience slight fluctuations or forecast errors, particularly during market volatility (e.g., seasonal demand changes, economic conditions). However, these errors will remain within an acceptable range and will not affect the overall market trend. The data may be obtained through large-sample surveys, sales data aggregation, and consumer behavior analysis, providing a reasonably accurate reflection of the market demand.

Drivers of Demand: The increase in pet numbers, the growing spending power of pet owners, innovations in pet food products (e.g., new types and qualities), and increasing pet owners' focus on pet health management are the key factors driving demand.

3.2.3. Pet Food Import/Export Rate

Assumption on Data Accuracy: We assume that the pet food import/export data comes from authoritative trade statistics organizations, customs data, or reports from industry associations. This data includes total import/export volumes, major importing/exporting countries and regions, product types, and other detailed information. We assume these data have been verified and accurately reflect the global and regional trade flows of pet food.

Margin of Error: We assume that the import/export data may be influenced by external factors such as international politics, economic conditions, and trade policies, which could cause some degree of error or delay. However, the overall trend of the data remains reliable and can be adjusted using appropriate methods to account for trade fluctuations.

Impact of International Markets: We assume that changes in pet food import/export volumes will have an impact on domestic supply and demand. For example, fluctuations in international market demand may cause domestic pet food prices to fluctuate or affect the supply of raw materials for pet food manufacturers.

3.2.4. Data Timeliness and Sustainability

Timeliness Assumption: We assume that all data is based on the most recent statistical periods (such as the latest annual or quarterly data). Considering the fast-changing market environment and consumer behavior, all data is regularly updated to ensure it reflects the current market situation.

Sustainability Assumption: We assume that the methods for data collection and analysis are sustainable in the long term, and can adapt to changes in the pet industry in the future.3.2.5. Assumptions on Data Integration and Consistency

Diversity and Integration of Data Sources: We assume that the data sources are diverse and have been effectively integrated, ensuring consistency across different sources. For instance, data on the demand for pet food and import/export rates may come from various channels, but through a unified data model and analysis methods, we can ensure coordination and consistency between these datasets.

Assumption of Analytical Validity: During the analysis process, we assume that the statistical models and forecasting methods (such as regression analysis, time series analysis, market segmentation, etc.) are appropriate and capable of handling correlations between different data points. Additionally, these methods consider the various factors influencing market demand, ensuring the validity and reliability of the analysis.

3.3Solution and Result

3.3.1Question one：

Table 1.Specific quantity changes of various pets from 2019 to 2023

Pet/year	2023	2022	2021	2020	2019
cat	6980	6536	5806	4862	4412
dog	5175	5119	5429	5222	5503
Aquatic animals	1530	1600	1650	1700	1780
crawlers	520	540	560	580	600
rodents	1080	1120	1150	1200	1250
bird	880	900	930	960	980

[2]

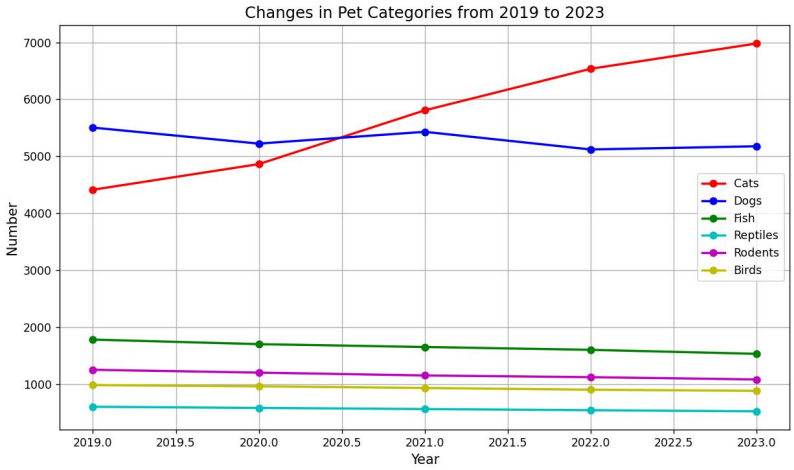

Figure 1 Change in Categories from 2019 to 2023

Table 2 Predicted number of pet cats and dogs from 2025-2026

Cat and Dog Population Forecast(in ten-thousands)	2024	2025	2026
Cat Population	7543.0615	8105.9709	8668.8803
Dog Population	5025.9097	4877.0800	4728.2504

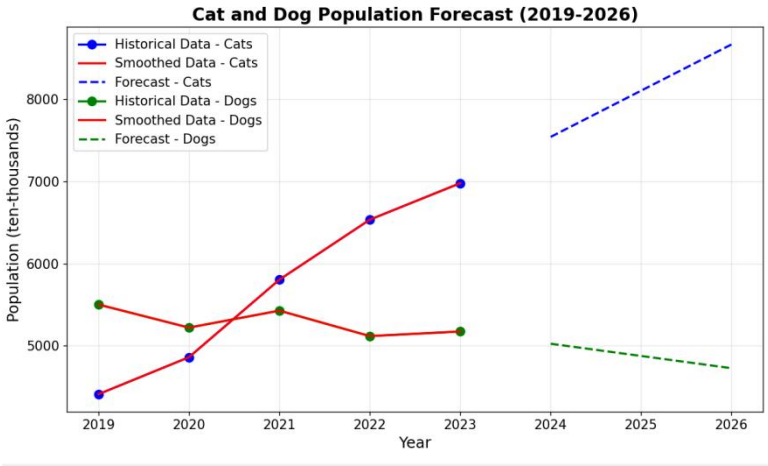

Figure 2 Cat and Dog Population Forecast(2019-2026)

The chart shows that the predicted number of pet cats in China is increasing year by year, while the number of pet dogs in China is decreasing year by year. With the continuous growth of the number of pet cats, it can be foreseen that the demand for pet cat related products and services in the market will increase. The pet cat care, food, toy and other industries need to adapt to this growth trend. Although the number of pet dogs has slightly decreased, due to its large base, it still holds a place in the market. Therefore, services and products related to pet dogs will remain an important market in the coming years and should not be rapidly reduced.

Table 3 Pet consumption from 2019 to 2023

Pet consumption/year	2023	2022	2021	2020	2019
Total consumption（trillion yuan）	3924	3117	2489	2058	2005
Pet medical consumption（trillion yuan）	1726	1216	881	655	483

[3]

Table 4 Pet_Food and Pet_ Medical population forecast

	2024	2025	2026
Pet_Food population forecast	830.31	886.88	943.45
Pet_Medical population forecast	2235.58	2745.34	3255.10

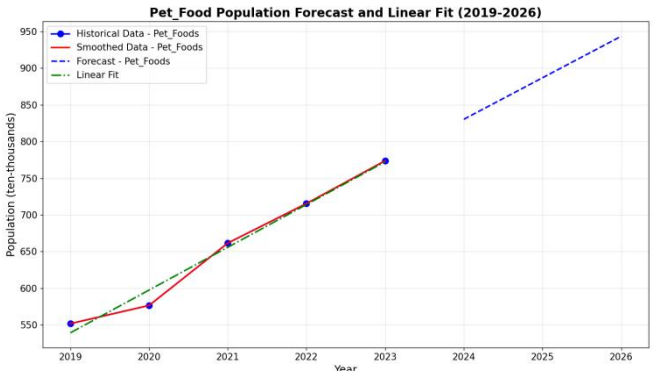

Figure 3 Pet_Food Population Forecast and Linear Fit(2019-2026)

The forecast data for the pet food market shows that the food model has an alpha optimal value of 1.00 and a beta optimal value of 1.00, with MSN_{min}=0.0091. The linear regression equation is y=2.57x-10329.65. The fitting degree R^2 is 0.9843.

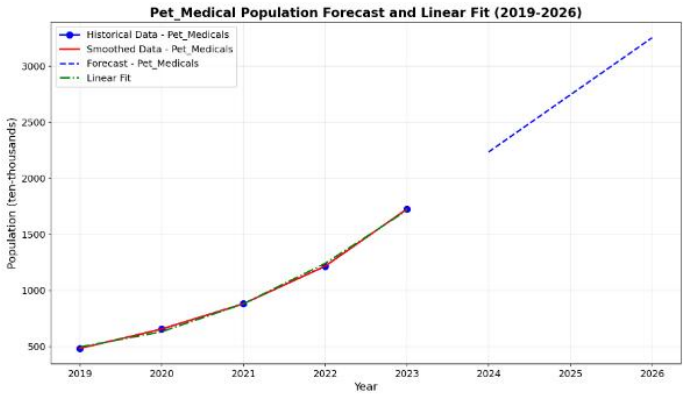

Figure 4 Pet_Medical Population Forecast and Linear Fit(2019-2026)

According to the predictions from our team's model, the optimal alpha for the medical model is 1.00, the optimal beta is 0.51, and at this point, the MSN is at its minimum of 0.0007. The linear regression equation is $y=56.07x^2 - 226336.01x - 226336.01$. The goodness of fit R^2 is 0.9985.

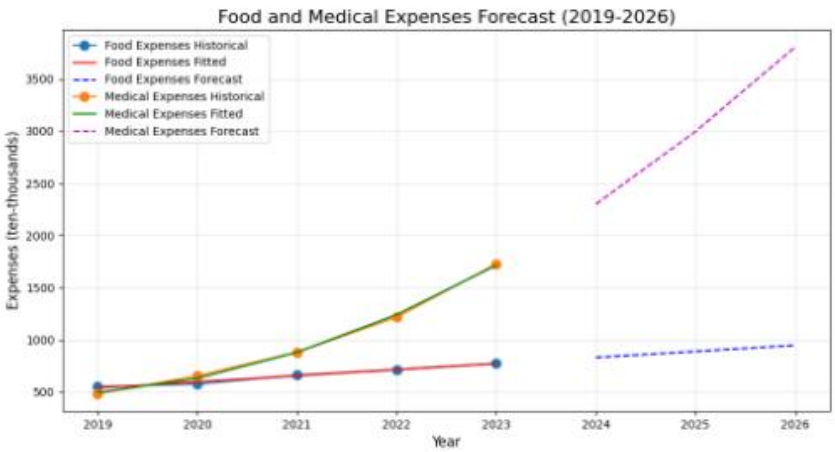

Figure 5 Food and Medical Expenses Forecast(2019-2026)

Pet food and medical expenses: Pet food expenses have increased from 55.174 billion yuan to 77.374 billion yuan, with an average annual increase of 5.829 billion

yuan (linear trend), reflecting the growing demand for quality and variety in pet food among pet owners; pet medical expenses have increased from 48.3 billion yuan to 172.6 billion yuan, with accelerated annual growth (consistent with a quadratic growth trend), indicating that pet owners are increasingly concerned about their pets' health, and the medical market has enormous potential.

3.3.2 Question two：

Table 5 Number of pets(cat and dog) in various countries worldwide(2019-2023)

Global pet population (cats and dogs) including forecasts	pet/year	2023	2022	2019	2018	2017
USA	cat	7380	9420	9420	6500	9420
	dag	8970	8970	8970	8500	8970
France	cat	1490	1510	1510	1490	1300
	dog	760	750	750	775	740
Germany	cat	1520	1670	1670	1570	1470
	dog	1060	1030	1030	1070	1010
China	cat	6980	6536	5806	4862	4412
	dog	5175	5119	5429	5222	5503

First, model each group of data based on the given data, perform data analysis to fit France and China's pet data to a simple linear regression, and predict for the next three years. Draw the following images:

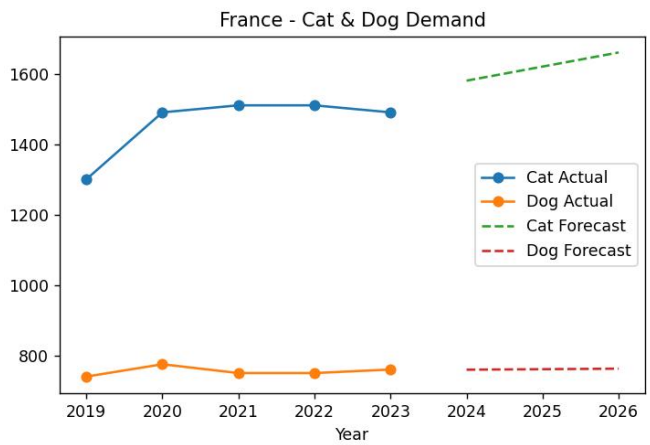

Figure 6 France-Cat & Dog Demand

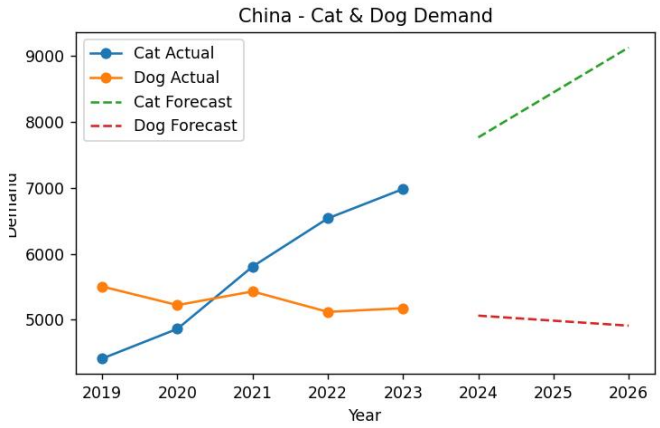

Figure 7 China-Cat & Dog Demand

German pet dogs and pet cats fit quadratic and cubic regression equations respectively, and after fitting, the graph shows a decrease in the number of pet cats and an increase in the number of pet dogs in Germany.

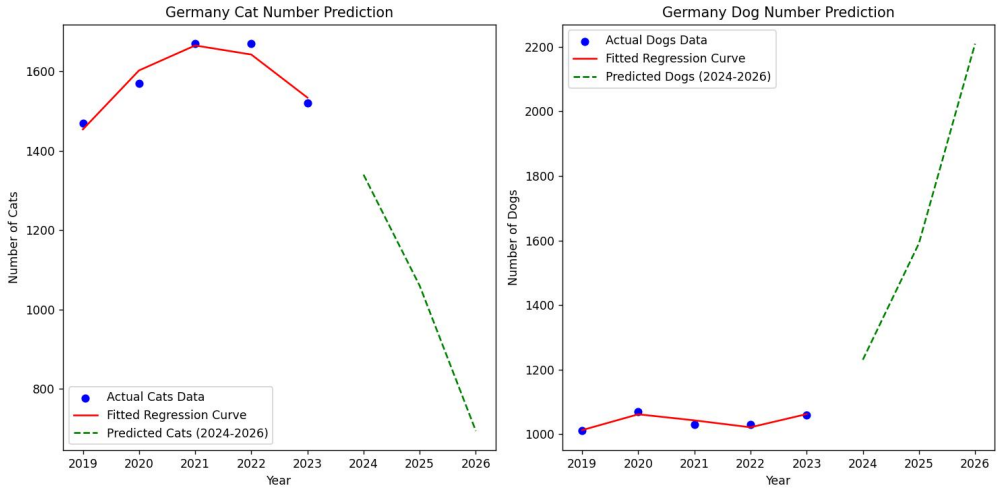

Figure 8 Germany Cat and Dog Number Prediction

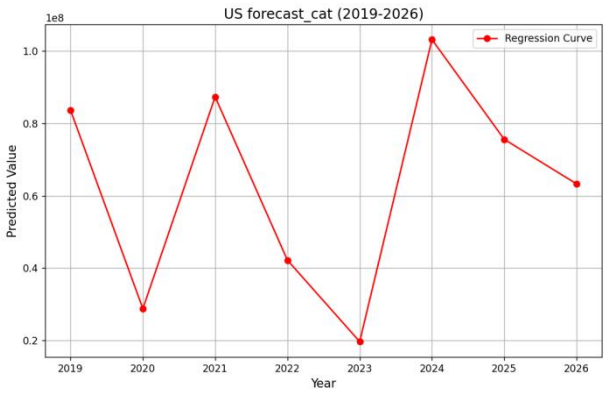

Figure 9 US forecast_cat(2019-2026)

Table 6 Relative error of regression method

Regression method frequency	Total relative error
1	0.7286

2	0.7101
3	0.2642
4	0.0000

The error for the third regression is already very small, and its performance on the test set is relatively good. Therefore, a regression of three times is a more suitable choice, as it maintains a low error while avoiding over fitting.

Table 7 Predicted global Dog and Cat food market demand

Year/Forecast global demand (in billion USD)	dog food market	cat food market
2021	1075.83	1102.31
2022	1180.18	1238.99
2023	1294.66	1392.63
2024	1420.24	1565.31
2025	1558.01	1759.41
2026	1709.13	1977.58

[4]

From the data, the demand in the dog food market shows a significant growth trend every year. This growth reflects the increase in the number of pet breeding, the rising attention to pet health among consumers, and the diversification of pet food products. The demand in the cat food market also exhibits a continuous growth trend, with an increase slightly higher than that of the dog food market.

Table 8 Dog and Cat food market demand slope

demand slope(billion USD per year)	Dog food market	Cat food market
	126.45	174.58

From the data, both the dog food market and the cat food market show a strong growth trend. Considering the global socioeconomic changes, the upgrading of pet consumption, and the tightening of environmental protection policies, the future pet food market is expected to continue to grow and may develop towards more sustainable and healthier directions.

3.3.3Question three：

Table 9 The production ,export value in China and global market size of pet food(2019-2023)

Project/year	2019	2020	2021	2022	2023
Total value of pet food production in China/billion	440.7	727.3	1554	1508	2793
Total export value of pet food from China/USD 100 mn	154.1	9.8	12.2	24.7	39.6

Global pet market size	1136.37	1236.37	1349.42	1409	1339

[4]

Table 10 CGAR of China's pet food production,export value and global market size

CGAR/project	Total value of pet food production in China/billion	Total export value of pet food from China/USD 100 mn	Global market size
	58.67%	-28.80%	4.19%

From the specific data of production, export value, and the global pet market size over the past five years, it can be seen that overall, production and market size have been steadily increasing, while the export value experienced a sudden decline in 2019 and has since gradually started to rise, albeit with a small growth rate that by 2023 had not yet recovered to its previous levels.

Looking at the CGAG (Compound Annual Growth Rate) over the past five years, it is evident that both production and market size have been on a general upward trend, whereas the CGAR value for export value being negative indicates that over these five years, the export value may have been consistently decreasing or there were years with sharp declines.

3.3.3.1Establishment and solution of Model Ⅰ (Exponential Smoothing Model)

Table 11 Measurement of actual and predicted values of pet food production,exports and global market size based on exponential smoothing method(2019-2026)

year	production/ billion	export value/ USD100mn	Market size/USD100 mn	Predicted production	Predicted export value	Predicted market size
2019	440.7	21.3	1136.7			
2020	727.3	9.8	1236.7			
2021	1554.0	12.2	1349.42			
2022	1508.0	24.7	1409.00			
2023	2793.0	39.6	1339.00			
2024				3031.95	54.5	1467.45
2025				3570.45	69.4	1525.26
2026				4108.94	84.3	1583.08

[5]

Production: Level = 0.1513, Trend = N/A ， Production Forecast: Y_t = 0.1513

Export: Level = 1.0000, Trend = N/A ， Export Forecast: Y_t = 1.0000

Market Size: Level = 0.0000, Trend = N/A ， Market Size Forecast: Y_t = 0.0000

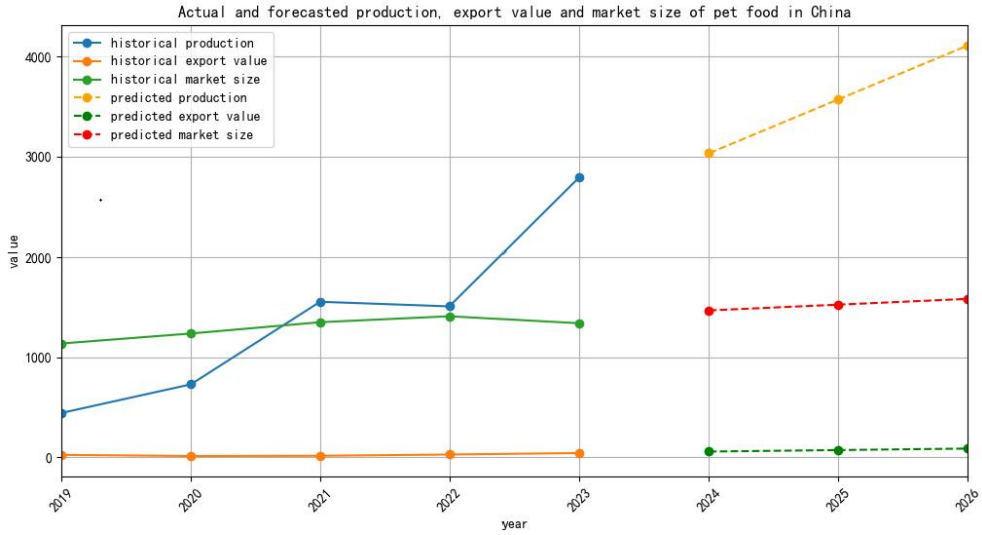

Figure 10 Actual and predicted production,export value and market size of pet food in China(method 1)

3.3.3.2Establishment and solution of Model II (Ridge Regression Model based on Polynomial Regression)

Table 12 Ridge regression based on multiple regression for predicting pet food production,export value and global market size(2024-2026)

year/project	Predicted production/billion	Predicted export value/USD100mn	Predicted market size/USD100mn
2024	3033.90	29.25	1451.50
2025	3577.00	31.82	1503.95
2026	4120.10	34.40	1556.39

Production Model Equation: y = 318.40 + (543.10) * x^1

Exports Model Equation: y = 16.37 + (2.58) * x^1

Market Model Equation: y = 1189.27 + (52.45) * x^1

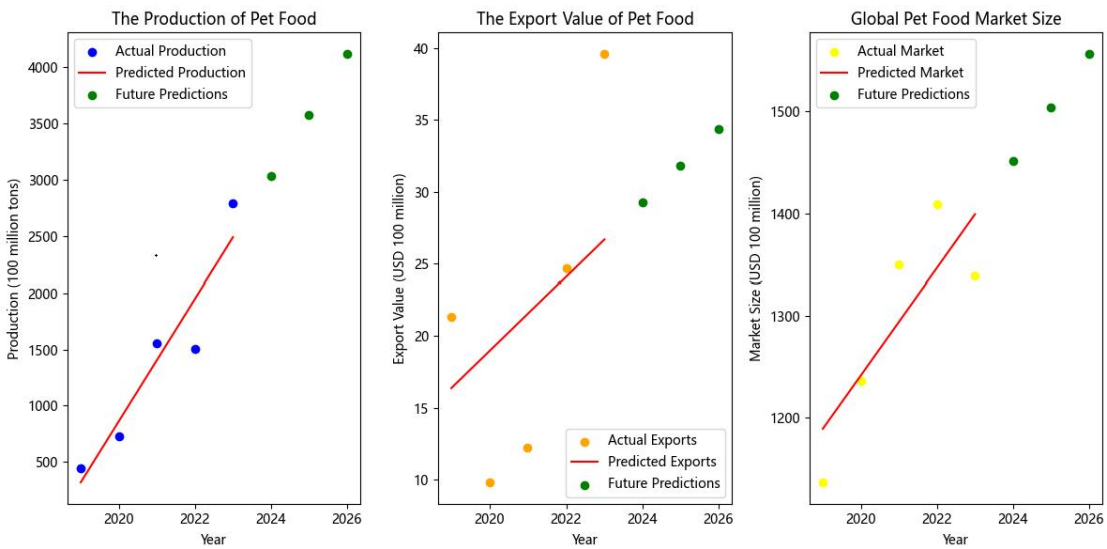

Figure 11 Actual and predicted production,export value and market size of pet food in China(method 2)

3.3.3.3Error Analysis:

Table 13 yield error analysis of ridge regression model based on polynomial regression

Project/year	2021	2022	2023
Actual value	1554	1508	2793
Predicted value	1172.01	1795.16	2650.90
Relative error	24.58%	19.04%	5.09%
Absolute error	381.99	287.16	142.1

Table 14 export value error analysis of ridge regression model based on polynomial regression

项目/年份	2021	2022	2023
Actual value	12.2	24.7	39.6
Predicted value	13.24	22.64	40.33
Relative error	8.52%	8.34%	1.84%
Absolute error	1.04	2.06	0.73

Based on the aforementioned predictive data and error analysis, China's pet food industry has achieved rapid growth over the past few years. Moreover, from the predictive values, it can be seen that the industry will maintain a good momentum of development in the coming three years. With the global pet food market size over the past five years and forecasts for the next three years, the global market size is continuously expanding, market demand is increasing, consumers' pursuit of high-quality products, and the expansion of e-commerce channels indicate that the production and export value of pet food will continue to rise.

3.3.4 Question 4：

Table 14 The average tariff of the United States on China(2019-2023)

The average tariff of the United States on China		
year	Tax rate(%)	Standard value
2019	17.5	0.0500
2020	21.3	0.0609
2021	19	0.0543
2022	19	0.0543
2023	19	0.0543

In 2019, tariffs increased due to the implementation of the U.S. policy to impose tariffs on Chinese goods. By 2020, the tariff rate had risen to 21.3%, mainly influenced by the intensification of the trade war and the progress of negotiations between the two sides. From 2021 to 2023, the tariff rate stabilized around 19%, indicating that since 2021, there has been no significant change in the tariff rate, suggesting that the trade war between the

U.S. and China has eased during this period.

Looking at the changes in the standard values, the relatively small changes in the standard values suggest that while the tariff rate increased in 2020, there may have been adjustments in the application of the rates across different goods or sectors. The standard values remained unchanged from 2021 to 2023, indicating that despite the stable tariff rates during these years, the actual application of the rates may not have significantly increased or decreased, possibly related to specific types of goods or their values.

Table 15 Predicted tax rates for the next few years

Predicted tax rates for the next few years	2024	2025	2026
Decision Tree Regression Prediction	19	19	19
Random Forest Regression Prediction	19	19	19
ARIMA Model Prediction	19.5153	19.2042	19.3920

Decision Tree Regression and Random Forest Regression: The predictions from these two models are very similar, both indicating that the tax rate will remain unchanged for the next few years (19). Although these regression models can capture some patterns in the data, they lack sensitivity to trends and seasonal changes when dealing with time series data, resulting in relatively simple and less dynamic predictions.

ARIMA Model: In contrast, the ARIMA model takes into account the dynamic changes in time series data and is able to identify subtle fluctuations and trends in tax rates. Its predictions are more in line with the patterns that may occur in real-world time series data, especially in modeling long-term trends and short-term fluctuations more accurately.

Table 16 MES & MAS of two kinds of regression model

	MSE	MAE
Decision Tree Regression Model Errors	0.00	0.00
Random Forest Regression Model Errors	0.63	0.58

Set the order of the ARIMA model to (1, 1, 1), perform differencing and forecasting.

Use the forecast() method to predict the tax rate for the next 3 years.

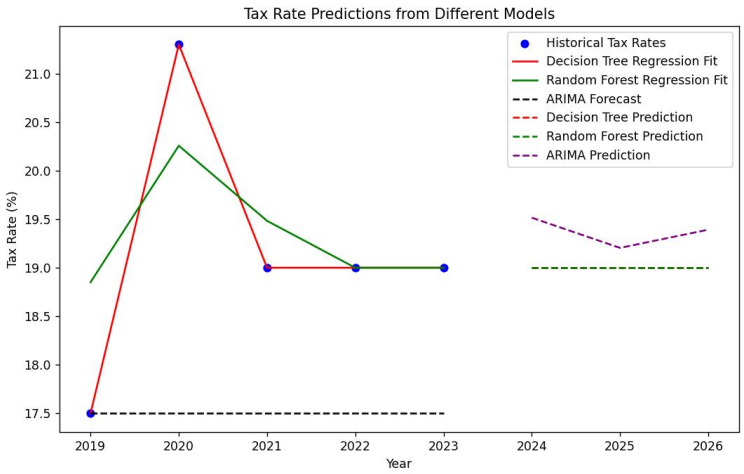

Figure 12 Tax rate predictions from different models

If the trend of tax rates over the next few years remains relatively stable without significant fluctuations, the predictions from decision tree and random forest models (19) may be reasonable. If there is a potential trend or cyclical fluctuation in the tax rate data, the predictions from the ARIMA model (19.515, 19.204, 19.392) would be more reasonable, as it can better reflect the dynamic changes in tax rates.

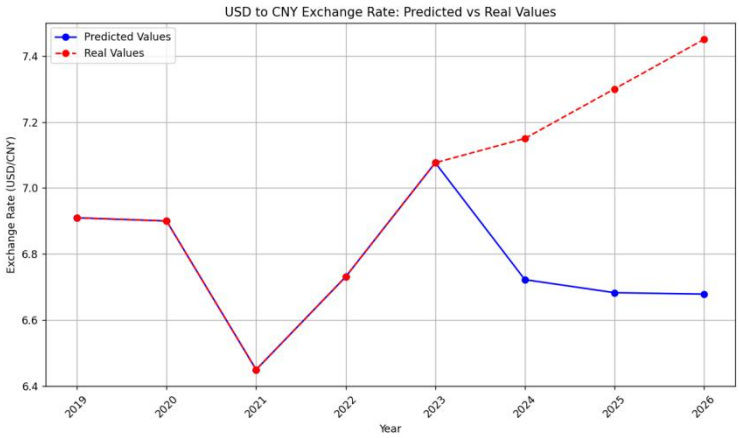

Figure 13 USD to CNY exchange rate:predicted vs real values

The predicted exchange rates show a slight downward trend from 6.72 in 2024 to 6.68 in 2025 and 2026. This indicates that, according to the model, the value of the Chinese Yuan (CNY) is expected to strengthen very slightly against the US Dollar (USD) in the coming years. The change is minimal, suggesting a relatively stable exchange rate in the near future.

Exchange rates are highly influenced by various factors, including monetary policy, interest rates, trade balances, and global events. If you are using these predictions for real-world decisions, it is important to compare them with expert economic forecasts or external data sources that incorporate broader economic indicators.

For instance, if there's an expectation of a U.S. economic slowdown or if China is expected to take actions that could influence the yuan's value (e.g., changes in monetary policy, trade negotiations, etc.), the model’s prediction might be inaccurate or too simplistic.

Short-term Focus: The model is trained on a small dataset, making its ability to predict

over several years more prone to error. A larger dataset, ideally spanning many more years and incorporating a variety of economic indicators, would likely improve prediction accuracy.

Volatility: LSTM models can struggle to predict sudden changes or high volatility in financial markets, as they are better suited for predicting smooth, gradual trends.

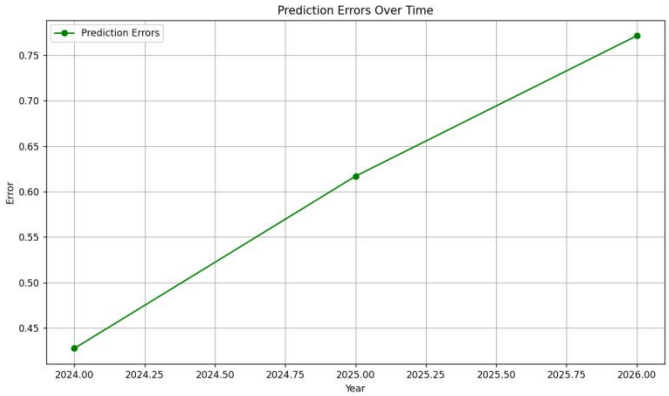

Figure 14 prediction error over time

The predicted USD/CNY exchange rates show a stable trend, with a slight decrease over the next few years. However, these predictions should be viewed with caution as they are based on limited historical data and do not incorporate broader economic factors that could influence exchange rates. To improve accuracy, expanding the data set and considering external economic factors would be beneficial.

3.4Analysis of the Result

3.4.1

The pet food market is expected to continue growing, but at a slower rate compared to previous years. According to the annual growth rate from 2024 to 2026, the growth is relatively stable, indicating that the demand for pet food remains strong, but not as rapid as in the past few years. As the pet industry in China matures, the pet food market may enter a period of steady growth, with a gradual slowdown in growth rates. Despite this, pet food remains one of the most basic and stable expenditure categories in pet consumption, with consumers' demand for high-quality pet food continuing to increase, especially in terms of nutrition, health, and natural foods.

Based on predictive data and error analysis, China's pet food industry has achieved rapid growth over the past few years. At the same time, it is evident from the forecast values that the next three years will maintain a good momentum of development. Looking at the global pet food market size over the past five years and the forecast for the next three years, the global market size and demand continue to increase. Consumers' pursuit of high-quality products and the expansion of e-commerce channels indicate that the production and export value of pet food will continue to rise.

The pet healthcare market is expected to continue to grow rapidly in the coming years. The average annual growth rate of the pet healthcare market is 21.5%, indicating that the demand for pet healthcare services will continue to rise quickly. As consumers pay more attention to their pets' health issues, the demand for high-end pet healthcare services (such as genetic testing, professional diagnosis and treatment, etc.) may further increase, driving the expansion of the entire healthcare consumption market.

3.4.2

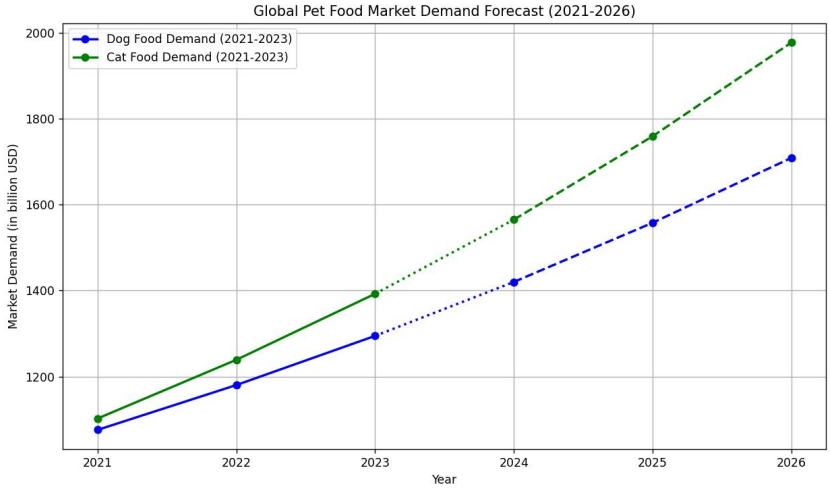

Figure 15 Global pet food market demand forecast(2021-2026)

Steady Growth: The growth trend of both markets is very clear, both showing a stable growth trend. It is expected that in the coming years, with the gradual development of global pet culture, market demand will continue to increase.

Faster Growth in Cat Food: Although the dog food market has a larger base, the growth rate of the cat food market is slightly higher, possibly due to the increase in the number of cats and the trend towards diversification of cat food products.

Market Expansion: This indicates the globalization trend of the pet industry, especially in the context of the rise of the middle class, accelerated urbanization, and pets being considered as family members, market demand will further expand.

Table 17 Predicted dog & cat food global market demand

Year/Forecast global demand (in billion USD)	dog food market	cat food market
2021	1075.83	1102.31
2022	1180.18	1238.99
2023	1294.66	1392.63
2024	1420.24	1565.31
2025	1558.01	1759.41
2026	1709.13	1977.58

From the data, the demand in the dog food market shows a significant growth trend every year. From 2021 to 2026, the demand increased from $107.583 billion to $170.913 billion, with an average annual growth of about 12-15%. This reflects an increase in the number of pet breeding, rising consumer attention to pet health, and the diversification of pet food products. The demand in the cat food market also exhibits a continuous growth trend, with the overall demand increasing from $110.231 billion in 2021 to $197.758 billion in 2026, with an average annual growth of about 14-18%. This increase is slightly higher than that of the dog food market. The popularity of cats as pets increases year by year, especially in urban areas, where cats are more suitable for living in small spaces compared to dogs,

especially among single and younger people.

Table 18 Dog & Cat food market demand slope

demand slope(billion USD per year)	Dog food market	Cat food market
	126.45	174.58

From the data, both the dog food market and the cat food market show a strong growth trend. The growth of the cat food market is slightly faster, which is related to the popularity of cats as pets and the increasing demand for high-quality food. Combining global socioeconomic changes, the upgrading of pet consumption, and the tightening of environmental protection policies, the future pet food market will continue to develop and may move towards a more sustainable and healthier direction.

3.4.3 **Results**

From the above forecast data and error analysis, it can be concluded that the pet food industry in our country has achieved rapid growth in the past few years, and at the same time, it can also be seen from the forecast value that the next three years will still maintain a good momentum of development. With the size of the global pet food market in the past five years and the forecast of the market size in the next three years, the global market size is expanding, the market demand is increasing, consumers' pursuit of high-quality products and the expansion of e-commerce channels show that the output and export volume of pet food will continue to rise.

3.4.4

Policy environment and International trade Tariffs and trade policies: With the continuous adjustment of tariff policies in European and American countries, especially in the import of pet food, it may cause some pressure on the export of pet food in China. European and US tariffs on Chinese pet food are likely to remain at around 20 per cent, potentially affecting demand in price-sensitive markets.

Domestic policy support: The Chinese government's policy support for the pet industry has been gradually strengthened, including improving the management of the industrial chain and improving food safety standards. This provides support for the sustainable development of the industry.

Exchange rate fluctuation: The fluctuation of RMB exchange rate will have a certain impact on exports. A weaker yuan could make Chinese pet food more price competitive in the international market, but it could also lead to higher costs for imported raw materials.

Global market changes: The demand for Chinese goods in Europe and the United States is affected by multiple factors, including economic growth, consumer preferences, and cultural differences. It is expected that the European and American markets will continue to be important export destinations for Chinese pet food, but other emerging markets (such as Southeast Asia, the Middle East, etc.) may gradually rise.

Sustainable Supply Chain: With the growing global concern for sustainable development, consumers are increasingly concerned about whether pet food is sourced from environmentally friendly and environmentally friendly sources. Pet food companies may need to put more emphasis on green sourcing and reducing their carbon footprint.

Alternative protein sources: Alternative proteins such as insect and plant proteins may become an important part of pet food production in the future. Many brands have started to introduce foods based on these alternative proteins, in line with environmental and health trends.

Pet humanization trend: Pets are increasingly viewed as members of the family, and consumers are willing to invest more in their health and well-being. With the improvement of consumer requirements for pet food quality, brand attention to health, nutrition, safety and other factors will be more prominent.

Personalization and customization: Pet food is gradually becoming personalized and customized to meet the needs of different pets. For example, providing tailored food based on the pet's age, weight, health status, etc.

Domestic competition: With the expansion of the market scale, more and more companies enter the pet food industry, and the competition is increasingly fierce. Domestic brands may need to stand out through innovation, product differentiation, branding, etc.、

International competition: Large European and American pet food companies (such as Nestle Purina, Mars Petcare, etc.) are still dominant, and they may influence the Chinese market through price advantages and channel advantages.

Ⅳ.Conclusions

As the pet industry continues to mature, the demand for pet food and medical services will further grow. The overall market has shown a strong growth trend and is expected to maintain a healthy expansion rate of about 30% per year in the next few years.

If the tariff policy remains stable, it is expected to have some suppression on exports, especially in price-sensitive markets such as Europe and the United States. However, tariff policy changes are small (assuming to stay within 20%) and growth in the domestic market remains the main driver.

Policy support will help improve the overall food safety level and brand competitiveness of the industry.

The yuan exchange rate is expected to stabilize between 6.7 and 6.8 between 2024 and 2026. A small depreciation may benefit exports, but the pressure from rising costs will also challenge profitability.

By 2026, 20% to 30% of pet food is expected to use alternative protein sources or meet sustainable production standards, and the market demand for eco-friendly and healthy products will drive this trend.

It is expected that the personalized and customized pet food market will account for a certain percentage (5%-10%) by 2026, especially in the high-end market.

Domestic companies are likely to capture a larger share of the global market in 2024 to 2026 if they can make breakthroughs in product innovation, brand building and supply chain management.

Comprehensive prediction conclusion:

China pet food Market: It is expected that the pet food market in China will continue to grow from 2024 to 2026, with an average annual growth rate of 10% to 15%, and the market size may exceed 600 billion yuan by 2026.

Technology and sustainability: Technological innovation and sustainability will be key factors driving the growth of the industry, with 10%-20% of pet food expected to be made with eco-friendly or alternative proteins.

Exports and international trade: Tariff policies and exchange rate changes will put some pressure on international trade, especially in European and American markets, but with the rise of emerging markets, exports may be diversified.

Competitive landscape: Industry competition will intensify, and domestic enterprises need to enhance their competitiveness in innovation, brand building and internationalization.

Policy support: The policy will further support the standardized development of the pet food industry, especially in safety standards and green production methods.

Ⅴ.Future Work

Pet industry development characteristics

Pet Type Characteristics:

The rapid increase in the number of cats is a major trend, reflecting the importance of cats as companion pets for people living alone and urban families.

The slight decrease in the number of dogs may be related to the limitation of living space in big cities, the standardization of the government's dog raising policy, and the higher cost of raising dog owners.

Non-traditional pets (aquarium, reptile, rodent, bird) are growing steadily, indicating diversified pet demand.

Driven by market demand:

The continued growth in spending on pet food and healthcare indicates a growing demand for high-quality, functional pet food, as well as a growing concern among pet owners for their pets' health.

The continued growth in spending on pet food and healthcare indicates a growing demand for high-quality, functional pet food, as well as a growing concern among pet owners for their pets' health. The prediction of future policies usually relies on the analysis of various aspects, including the current economic situation, social development trends, technological advances, and changes in global markets. Here are a few key areas of predictions for possible future policies in China and globally:

1. Economic policy

Steady growth and innovation-driven: The Chinese government is expected to continue to implement steady growth policies, especially against the backdrop of increasing global economic uncertainty. Policy is likely to focus more on boosting economic growth by boosting consumption, increasing investment in infrastructure, and supporting high-tech industries and innovative businesses. More policies are expected to support emerging areas such as digital economy, green economy and smart manufacturing.

Tax cuts and fiscal support: Faced with downward pressure on the economy, further tax cuts or fiscal subsidies may be provided, especially for small and medium-sized enterprises and technological innovation. The implementation of such policies is expected to promote

the vitality of enterprises and improve the overall consumption capacity of society.

Financial policy: It is likely to further open up the financial market and guide the development of the capital market to promote the injection of long-term funds and support private investment and innovation. Interest rate policy is likely to ease to boost consumption and investment.

2. Technology and Digital Transformation Policy

5G and artificial intelligence support: The government is expected to continue to introduce policies to support the development of cutting-edge technologies such as 5G, artificial intelligence and big data to boost the rapid growth of the digital economy. In particular, scientific and technological innovation will be the focus of policy support in manufacturing, education and medical care.

Data privacy and cybersecurity regulations: With the acceleration of the digitalization process, it is expected that the government will further introduce regulations on data privacy protection and cybersecurity, and strengthen the supervision of Internet companies to ensure that data security, consumer privacy protection and technological development are coordinated.

Industrial policy for scientific and technological innovation: In the next few years, the government will continue to promote the research and development of core technologies through financial support, R&D subsidies and tax breaks, especially in strategic emerging industries such as semiconductors, new energy and life sciences.

3.Social policy

Population and aging response: As China's aging problem becomes increasingly serious, more social security, pension services and family policies are expected to be introduced in the future. For example, tax incentives for young families may be introduced to encourage childbearing; Or increase support for an aging society, and push for reform of long-term care insurance and social welfare.

Education and labor market reform: In order to cope with the changes in labor market demand brought by future technological changes, education policies will focus on strengthening investment in vocational education and skill training to cope with the impact of intelligence and automation on the labor market. At the same time, policies may be introduced to encourage new types of work, such as flexible employment and remote working.

4. International trade and external policy

Trade liberalization and Global Supply chain Layout: China will continue to promote trade liberalization, especially under the framework of the Belt and Road Initiative, further expand the international market and strengthen cooperation with developing countries. At the same time, as the international situation changes, it may adopt a diversified international supply chain strategy to reduce dependence on the single market.

Further opening up of foreign investment policy: China may continue to open up its financial market and capital market to attract foreign investment. The government may further relax the foreign ownership limit to promote the investment and development of multinational companies in China.

Summary

Future policies will focus on promoting stable economic growth, green and sustainable development, scientific and technological innovation and digital transformation, and pay more attention to social governance and the improvement of social welfare. The core objectives of the policy are to respond to global change, strengthen domestic economic resilience, promote scientific and technological progress, and improve the quality of life of people. In the process, the government is likely to introduce more flexible, inclusive and forward-looking measures to address complex economic, social and environmental challenges.

Ⅵ. References

[1] China's pet food trade know how much -- CPPCC net

[2] Market Size and user insight Analysis of China's pet industry - Pet Industry Database

[3] China's pet medical industry: Analysis of the trend of user demand Upgrading and online and offline integration

[4] In-depth report on the pet industry: Overseas perspective, relatively resilient consumption, and enduring brand refreshing_Tencent News

[5] Analysis of the development status and market size of the global and Chinese pet food industry in 2021. The growth rate of the domestic market size is higher than that of the global _Forward-looking Trend-Forward-looking Industry Research Institute

[6] Figure 2 The number of pets and market size in China (2018 ~ 2021)

Ⅶ.Appendix

The code for the First question

```
#the first question
#---------p1.py---------------
# Pet Food Forecast
#----------------------------------
import numpy as np
import matplotlib.pyplot as plt

# Given data
years = np.array([2019, 2020, 2021, 2022, 2023])
pet_food = np.array([551.74, 576.50, 661.52, 715.44, 773.74])

# Function to calculate Mean Squared Error (MSE)
def calculate_mse(alpha, beta, data):
    # Initialize smoothing and trend values
    S = [data[0]]  # List to store smoothed values
    b = [data[1] - data[0]]  # List to store trend values

    # Recursive calculation of smoothed values and trend
    for t in range(1, len(data)):
        S_t = alpha * data[t] + (1 - alpha) * (S[-1] + b[-1])  # Smoothed value at time t
        b_t = beta * (S_t - S[-1]) + (1 - beta) * b[-1]  # Updated trend value at time t
        S.append(S_t)  # Add smoothed value to the list
        b.append(b_t)  # Add trend value to the list

    # Calculate predicted values and Mean Squared Error (MSE)
    predicted = S  # Predicted values are the smoothed values
    mse = np.mean((data - predicted) ** 2)  # Compute MSE between actual and predicted values
    return mse

# Test all possible combinations of alpha and beta
alpha_values = np.arange(0.1, 1.0, 0.001)  # Values for alpha from 0.1 to 1.0
beta_values = np.arange(0.1, 1.0, 0.001)   # Values for beta from 0.1 to 1.0

# Initialize variables to store the best parameters and MSE
best_mse_pet_food = float('inf')
best_alpha_pet_food = None
best_beta_pet_food = None

# Grid search for the best alpha and beta for the pet food data
for alpha in alpha_values:
```

```
        for beta in beta_values:
            mse_pet_food = calculate_mse(alpha, beta, pet_food)  # Compute MSE for current alpha, beta pair
            print("Matching ", alpha, " with ", beta, " in progress...")

            # Update best parameters if a better MSE is found
            if mse_pet_food < best_mse_pet_food:
                best_mse_pet_food = mse_pet_food
                best_alpha_pet_food = alpha
                best_beta_pet_food = beta

    # Output the best parameters and corresponding MSE
    print(f"Best alpha for pet_food = {best_alpha_pet_food:.2f}, Best beta for pet_food = {best_beta_pet_food:.2f}")
    print(f"Minimum MSE for pet_food = {best_mse_pet_food:.4f}")

    # Assign best alpha and beta values for further calculations
    alpha_pet_food = best_alpha_pet_food
    beta_pet_food = best_beta_pet_food

    # Initialize smoothed and trend values for the pet food data
    S_pet_food = [pet_food[0]]
    b_pet_food = [pet_food[1] - pet_food[0]]

    # Perform smoothing and trend estimation
    for t in range(1, len(pet_food)):
        S_t = alpha_pet_food * pet_food[t] + (1 - alpha_pet_food) * (S_pet_food[-1] + b_pet_food[-1])  # Smoothed value
        b_t = beta_pet_food * (S_t - S_pet_food[-1]) + (1 - beta_pet_food) * b_pet_food[-1]  # Trend estimation
        S_pet_food.append(S_t)  # Append smoothed value
        b_pet_food.append(b_t)  # Append trend value

    # Forecast pet food population for the next three years
    future_years = np.array([2024, 2025, 2026])
    future_pet_food = [S_pet_food[-1] + k * b_pet_food[-1] for k in range(1, len(future_years) + 1)]  # Linear extrapolation

    # Output the forecast for the next three years
    print("Pet_Food population forecast for the next three years (in ten-thousands):", future_pet_food)

    # Perform linear regression (least squares fit) to get the linear trend
    coefficients = np.polyfit(years, pet_food, 1)  # Fit a first-degree polynomial (linear)
```

```
linear_fit = np.polyval(coefficients, years)  # Calculate the linear fit values

# Calculate R-squared value (goodness of fit)
ss_total = np.sum((pet_food - np.mean(pet_food)) ** 2)  # Total sum of squares
ss_residual = np.sum((pet_food - linear_fit) ** 2)  # Residual sum of squares
r_squared = 1 - (ss_residual / ss_total)  # R-squared

# Print linear fit results
print(f"Linear fit equation: y = {coefficients[0]:.2f}x + {coefficients[1]:.2f}")
print(f"R-squared value: {r_squared:.4f}")

# Plot the results
plt.figure(figsize=(12, 7))

# Plot original data and smoothed curve
plt.plot(years, pet_food, 'o-', label='Historical Data - Pet_Foods', color='blue', markersize=8, linewidth=2)
plt.plot(years, S_pet_food, 'r-', label='Smoothed Data - Pet_Foods', linewidth=2)

# Plot future forecast values
plt.plot(future_years, future_pet_food, 'b--', label='Forecast - Pet_Foods', linewidth=2)

# Plot the linear fit
plt.plot(years, linear_fit, 'g-.', label='Linear Fit', linewidth=2)

# Add title and axis labels
plt.title('Pet_Food Population Forecast and Linear Fit (2019-2026)', fontsize=16, fontweight='bold')
plt.xlabel('Year', fontsize=14)
plt.ylabel('Population (ten-thousands)', fontsize=14)
plt.xticks(np.arange(2019, 2027), fontsize=12)
plt.yticks(fontsize=12)
plt.legend(fontsize=12)
plt.grid(alpha=0.3)
plt.tight_layout()  # Ensure layout is tight
plt.show()  # Display the plot

#-----------------------------------------------------------------------code-----------------------------------------#
import matplotlib.pyplot as plt

# Data
years = [2019, 2020, 2021, 2022, 2023]
cats = [4412, 4862, 5806, 6536, 6980]
```

```
dogs = [5503, 5222, 5429, 5119, 5175]
fish = [1780, 1700, 1650, 1600, 1530]
reptiles = [600, 580, 560, 540, 520]
rodents = [1250, 1200, 1150, 1120, 1080]
birds = [980, 960, 930, 900, 880]

# Create the plot
plt.figure(figsize=(10, 6))

plt.plot(years, cats, label='Cats', marker='o', color='r', linestyle='-', linewidth=2)
plt.plot(years, dogs, label='Dogs', marker='o', color='b', linestyle='-', linewidth=2)
plt.plot(years, fish, label='Fish', marker='o', color='g', linestyle='-', linewidth=2)
plt.plot(years, reptiles, label='Reptiles', marker='o', color='c', linestyle='-', linewidth=2)
plt.plot(years, rodents, label='Rodents', marker='o', color='m', linestyle='-', linewidth=2)
plt.plot(years, birds, label='Birds', marker='o', color='y', linestyle='-', linewidth=2)

# Set title and labels
plt.title("Changes in Pet Categories from 2019 to 2023", fontsize=14)
plt.xlabel("Year", fontsize=12)
plt.ylabel("Number", fontsize=12)

# Display grid
plt.grid(True)

# Add legend
plt.legend()

# Display the plot
plt.tight_layout()
plt.show()

#-----------------------------------------------------------------------------code-----------------------------------------#

#---------------------------------
# Pet Food Forecasting
#---------------------------------
import numpy as np
import matplotlib.pyplot as plt

# Given data: Years and corresponding Pet Food Population (in ten-thousands)
years = np.array([2019, 2020, 2021, 2022, 2023])
pet_food = np.array([551.74, 576.50, 661.52, 715.44, 773.74])

```

```python

# Function to calculate Mean Squared Error (MSE)
def calculate_mse(alpha, beta, data):
    # Initialize the smoothed values and trend lists
    S = [data[0]]  # Smoothed values
    b = [data[1] - data[0]]  # Trend values

    # Recursive calculation of smoothed values and trend
    for t in range(1, len(data)):
        S_t = alpha * data[t] + (1 - alpha) * (S[-1] + b[-1])  # New smoothed value
        b_t = beta * (S_t - S[-1]) + (1 - beta) * b[-1]  # New trend value
        S.append(S_t)
        b.append(b_t)

    # Calculate the predicted values and MSE
    predicted = S  # Predicted values are the smoothed values
    mse = np.mean((data - predicted) ** 2)  # Mean Squared Error
    return mse


# Testing all possible combinations of alpha and beta
alpha_values = np.arange(0.1, 1.0, 0.001)
beta_values = np.arange(0.1, 1.0, 0.001)

# Initialize variables to store the best parameters and MSE
best_mse_pet_food = float('inf')
best_alpha_pet_food = None
best_beta_pet_food = None


# Grid search to find the best alpha and beta for pet food
for alpha in alpha_values:
    for beta in beta_values:
        mse_pet_food = calculate_mse(alpha, beta, pet_food)  # Calculate MSE for current alpha and beta
        print("Matching ", alpha, " and ", beta, " in progress")
        # Update the best parameters if the MSE is smaller
        if mse_pet_food < best_mse_pet_food:
            best_mse_pet_food = mse_pet_food
            best_alpha_pet_food = alpha
            best_beta_pet_food = beta


# Output the best parameters and corresponding MSE
```

```python
print(f"Best alpha for pet_food = {best_alpha_pet_food:.2f}, Best beta for pet_food = {best_beta_pet_food:.2f}")
print(f"Minimum MSE for pet_food = {best_mse_pet_food:.4f}")


# Find the optimal alpha and beta for pet food
alpha_pet_food = best_alpha_pet_food
beta_pet_food = best_beta_pet_food

# Initialize smoothed values (S) and trends (b) for the forecast
S_pet_food = [pet_food[0]]
b_pet_food = [pet_food[1] - pet_food[0]]

# Apply the exponential smoothing method
for t in range(1, len(pet_food)):
    S_t = alpha_pet_food * pet_food[t] + (1 - alpha_pet_food) * (S_pet_food[-1] + b_pet_food[-1]) # Update smoothed value
    b_t = beta_pet_food * (S_t - S_pet_food[-1]) + (1 - beta_pet_food) * b_pet_food[-1]  # Update trend value
    S_pet_food.append(S_t)
    b_pet_food.append(b_t)


# Predict the next 3 years
future_years = np.array([2024, 2025, 2026])
future_pet_food = [S_pet_food[-1] + k * b_pet_food[-1] for k in range(1, len(future_years) + 1)]

# Print the forecasted pet food population for the next three years
print("Pet_Food population forecast for the next three years (in ten-thousands):", future_pet_food)

# Linear fit (1st degree polynomial) for the data
coefficients = np.polyfit(years, pet_food, 1)  # First degree polynomial (linear fit)
linear_fit = np.polyval(coefficients, years)  # Generate the linear fit curve

# Calculate the goodness of fit (R-squared)
ss_total = np.sum((pet_food - np.mean(pet_food)) ** 2)  # Total sum of squares
ss_residual = np.sum((pet_food - linear_fit) ** 2)  # Residual sum of squares
r_squared = 1 - (ss_residual / ss_total)  # R-squared value

# Output the linear fit equation and R-squared value
print(f"Linear fit equation: y = {coefficients[0]:.2f}x + {coefficients[1]:.2f}")
print(f"R-squared value: {r_squared:.4f}")

```

```
# Plotting the results
plt.figure(figsize=(12, 7))

# Plot the original data and the smoothed exponential smoothing curve
plt.plot(years, pet_food, 'o-', label='Historical Data - Pet_Foods', color='blue', markersize=8, linewidth=2)
plt.plot(years, S_pet_food, 'r-', label='Smoothed Data - Pet_Foods', linewidth=2)

# Plot the forecasted values for the next years
plt.plot(future_years, future_pet_food, 'b--', label='Forecast - Pet_Foods', linewidth=2)

# Plot the linear fit curve
plt.plot(years, linear_fit, 'g-.', label='Linear Fit', linewidth=2)

# Add title and labels
plt.title('Pet_Food Population Forecast and Linear Fit (2019-2026)', fontsize=16, fontweight='bold')
plt.xlabel('Year', fontsize=14)
plt.ylabel('Population (ten-thousands)', fontsize=14)
plt.xticks(np.arange(2019, 2027), fontsize=12)
plt.yticks(fontsize=12)
plt.legend(fontsize=12)
plt.grid(alpha=0.3)
plt.tight_layout()

# Show the plot
plt.show()

#-----------------------------------------------------------------------code-----------------------------------------#

import numpy as np
import matplotlib.pyplot as plt

# Given data
years = np.array([2019, 2020, 2021, 2022, 2023])  # Years
cats = np.array([4412, 4862, 5806, 6536, 6980])   # Cat population data (in ten-thousands)
dogs = np.array([5503, 5222, 5429, 5119, 5175])   # Dog population data (in ten-thousands)


# Function to calculate Mean Squared Error (MSE)
def calculate_mse(alpha, beta, data):
    # Initialize smoothed values and trends
    S = [data[0]]  # List for smoothed values
```

```
    b = [data[1] - data[0]]  # List for trend values

    # Recursive calculation of smoothed values and trends
    for t in range(1, len(data)):
        S_t = alpha * data[t] + (1 - alpha) * (S[-1] + b[-1])  # Smoothed value
        b_t = beta * (S_t - S[-1]) + (1 - beta) * b[-1]  # Trend value
        S.append(S_t)
        b.append(b_t)

    # Calculate predicted values and Mean Squared Error (MSE)
    predicted = S  # Predicted values are the smoothed values
    mse = np.mean((data - predicted) ** 2)  # Calculate MSE
    return mse


# Test all possible combinations of alpha and beta
alpha_values = np.arange(0.1, 1.0, 0.001)  # Range of alpha values
beta_values = np.arange(0.1, 1.0, 0.001)   # Range of beta values

# Initialize variables to store the best parameters and MSE for cats
best_mse_cats = float('inf')
best_alpha_cats = None
best_beta_cats = None

# Initialize variables to store the best parameters and MSE for dogs
best_mse_dogs = float('inf')
best_alpha_dogs = None
best_beta_dogs = None

# Grid search for optimal alpha and beta for cats and dogs
for alpha in alpha_values:
    for beta in beta_values:
        mse_cats = calculate_mse(alpha, beta, cats)  # Calculate MSE for cats
        mse_dogs = calculate_mse(alpha, beta, dogs)  # Calculate MSE for dogs
        print("Testing alpha:", alpha, " and beta:", beta)

        # Update best parameters for cats if current MSE is lower
        if mse_cats < best_mse_cats:
            best_mse_cats = mse_cats
            best_alpha_cats = alpha
            best_beta_cats = beta

        # Update best parameters for dogs if current MSE is lower
        if mse_dogs < best_mse_dogs:
```

```
            best_mse_dogs = mse_dogs
            best_alpha_dogs = alpha
            best_beta_dogs = beta

# Output the best parameters and corresponding MSE for cats and dogs
print(f"Best alpha for cats = {best_alpha_cats:.2f}, Best beta for cats = {best_beta_cats:.2f}")
print(f"Minimum MSE for cats = {best_mse_cats:.4f}")

print(f"Best alpha for dogs = {best_alpha_dogs:.2f}, Best beta for dogs = {best_beta_dogs:.2f}")
print(f"Minimum MSE for dogs = {best_mse_dogs:.4f}")

# Compute smoothed data for cats using the best alpha and beta
alpha_cats = best_alpha_cats
beta_cats = best_beta_cats

S_cats = [cats[0]]  # Initialize smoothed values list
b_cats = [cats[1] - cats[0]]  # Initialize trend values list

for t in range(1, len(cats)):
    S_t = alpha_cats * cats[t] + (1 - alpha_cats) * (S_cats[-1] + b_cats[-1])  # Smoothed value
    b_t = beta_cats * (S_t - S_cats[-1]) + (1 - beta_cats) * b_cats[-1]  # Trend value
    S_cats.append(S_t)
    b_cats.append(b_t)

# Compute smoothed data for dogs using the best alpha and beta
alpha_dogs = best_alpha_dogs
beta_dogs = best_beta_dogs

S_dogs = [dogs[0]]  # Initialize smoothed values list
b_dogs = [dogs[1] - dogs[0]]  # Initialize trend values list

for t in range(1, len(dogs)):
    S_t = alpha_dogs * dogs[t] + (1 - alpha_dogs) * (S_dogs[-1] + b_dogs[-1])  # Smoothed value
    b_t = beta_dogs * (S_t - S_dogs[-1]) + (1 - beta_dogs) * b_dogs[-1]  # Trend value
    S_dogs.append(S_t)
    b_dogs.append(b_t)

# Forecast future populations for the next three years (2024-2026)
future_years = np.array([2024, 2025, 2026])  # Future years for forecast
future_cats = [S_cats[-1] + k * b_cats[-1] for k in range(1, len(future_years) + 1)]  # Future cat population forecast
future_dogs = [S_dogs[-1] + k * b_dogs[-1] for k in range(1, len(future_years) + 1)]  # Future dog population forecast

```

```
# Output forecast results
print("Cat population forecast for the next three years (in ten-thousands):", future_cats)
print("Dog population forecast for the next three years (in ten-thousands):", future_dogs)

# Plotting the historical and forecasted data
plt.figure(figsize=(10, 6))

# Historical data and smoothed data for cats and dogs
plt.plot(years, cats, 'o-', label='Historical Data - Cats', color='blue', markersize=8, linewidth=2)
plt.plot(years, S_cats, 'r-', label='Smoothed Data - Cats', linewidth=2)
plt.plot(future_years, future_cats, 'b--', label='Forecast - Cats', linewidth=2)

plt.plot(years, dogs, 'o-', label='Historical Data - Dogs', color='green', markersize=8, linewidth=2)
plt.plot(years, S_dogs, 'r-', label='Smoothed Data - Dogs', linewidth=2)
plt.plot(future_years, future_dogs, 'g--', label='Forecast - Dogs', linewidth=2)

# Add titles and labels
plt.title('Cat and Dog Population Forecast (2019-2026)', fontsize=16, fontweight='bold')
plt.xlabel('Year', fontsize=14)
plt.ylabel('Population (ten-thousands)', fontsize=14)
plt.xticks(np.arange(2019, 2027), fontsize=12)
plt.yticks(fontsize=12)
plt.legend(fontsize=12)
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()

#-----------------------------------------------------------------------code-----------------------------------------#
import numpy as np
import matplotlib.pyplot as plt

# Given data
years = np.array([2019, 2020, 2021, 2022, 2023])
cats = np.array([4412, 4862, 5806, 6536, 6980])
dogs = np.array([5503, 5222, 5429, 5119, 5175])

# Function to calculate Mean Squared Error (MSE)
def calculate_mse(alpha, beta, data):
    # Initialize the smoothed values and trend
    S = [data[0]]  # Smoothed values list
    b = [data[1] - data[0]]  # Trend values list
```

```python

    # Recursive calculation of smoothed values and trend
    for t in range(1, len(data)):
        S_t = alpha * data[t] + (1 - alpha) * (S[-1] + b[-1])
        b_t = beta * (S_t - S[-1]) + (1 - beta) * b[-1]
        S.append(S_t)
        b.append(b_t)

    # Calculate predicted values and MSE
    predicted = S  # Predicted values are the smoothed values
    mse = np.mean((data - predicted) ** 2)  # Compute Mean Squared Error
    return mse

# Test all possible combinations of alpha and beta
alpha_values = np.arange(0.1, 1.0, 0.001)
beta_values = np.arange(0.1, 1.0, 0.001)

# Initialize variables to store best parameters and MSE for cats
best_mse_cats = float('inf')
best_alpha_cats = None
best_beta_cats = None

# Initialize variables to store best parameters and MSE for dogs
best_mse_dogs = float('inf')
best_alpha_dogs = None
best_beta_dogs = None

# Perform grid search for cats and dogs
for alpha in alpha_values:
    for beta in beta_values:
        mse_cats = calculate_mse(alpha, beta, cats)
        mse_dogs = calculate_mse(alpha, beta, dogs)
        print(f"Testing alpha = {alpha}, beta = {beta}")

        # Update the best parameters for cats
        if mse_cats < best_mse_cats:
            best_mse_cats = mse_cats
            best_alpha_cats = alpha
            best_beta_cats = beta

        # Update the best parameters for dogs
        if mse_dogs < best_mse_dogs:
            best_mse_dogs = mse_dogs
            best_alpha_dogs = alpha
```

```
            best_beta_dogs = beta

# Output the best parameters and corresponding MSE
print(f"Best alpha for cats = {best_alpha_cats:.2f}, Best beta for cats = {best_beta_cats:.2f}")
print(f"Minimum MSE for cats = {best_mse_cats:.4f}")

print(f"Best alpha for dogs = {best_alpha_dogs:.2f}, Best beta for dogs = {best_beta_dogs:.2f}")
print(f"Minimum MSE for dogs = {best_mse_dogs:.4f}")

# Find the best alpha and beta for cats
alpha_cats = best_alpha_cats
beta_cats = best_beta_cats

S_cats = [cats[0]]
b_cats = [cats[1] - cats[0]]

for t in range(1, len(cats)):
    S_t = alpha_cats * cats[t] + (1 - alpha_cats) * (S_cats[-1] + b_cats[-1])
    b_t = beta_cats * (S_t - S_cats[-1]) + (1 - beta_cats) * b_cats[-1]
    S_cats.append(S_t)
    b_cats.append(b_t)

# Find the best alpha and beta for dogs
alpha_dogs = best_alpha_dogs
beta_dogs = best_beta_dogs

S_dogs = [dogs[0]]
b_dogs = [dogs[1] - dogs[0]]

for t in range(1, len(dogs)):
    S_t = alpha_dogs * dogs[t] + (1 - alpha_dogs) * (S_dogs[-1] + b_dogs[-1])
    b_t = beta_dogs * (S_t - S_dogs[-1]) + (1 - beta_dogs) * b_dogs[-1]
    S_dogs.append(S_t)
    b_dogs.append(b_t)

# Forecast for the next three years
future_years = np.array([2024, 2025, 2026])
future_cats = [S_cats[-1] + k * b_cats[-1] for k in range(1, len(future_years) + 1)]
future_dogs = [S_dogs[-1] + k * b_dogs[-1] for k in range(1, len(future_years) + 1)]

# Print forecast results
print("Cat population forecast for the next three years (in ten-thousands):", future_cats)
print("Dog population forecast for the next three years (in ten-thousands):", future_dogs)

```

```
# Plotting the data
plt.figure(figsize=(10, 6))

# Historical data and forecasted values
plt.plot(years, cats, 'o-', label='Historical Data - Cats', color='blue', markersize=8, linewidth=2)
plt.plot(years, S_cats, 'r-', label='Smoothed Data - Cats', linewidth=2)
plt.plot(future_years, future_cats, 'b--', label='Forecast - Cats', linewidth=2)

plt.plot(years, dogs, 'o-', label='Historical Data - Dogs', color='green', markersize=8, linewidth=2)
plt.plot(years, S_dogs, 'r-', label='Smoothed Data - Dogs', linewidth=2)
plt.plot(future_years, future_dogs, 'g--', label='Forecast - Dogs', linewidth=2)

# Add titles and labels
plt.title('Cat and Dog Population Forecast (2019-2026)', fontsize=16, fontweight='bold')
plt.xlabel('Year', fontsize=14)
plt.ylabel('Population (ten-thousands)', fontsize=14)
plt.xticks(np.arange(2019, 2027), fontsize=12)
plt.yticks(fontsize=12)
plt.legend(fontsize=12)
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()

#-----------------------------------------------------------------------code----------------------------------------#

import numpy as np
import matplotlib.pyplot as plt

# Data for different pet types over the years
years = np.array([2019, 2020, 2021, 2022, 2023])  # Years
cats = np.array([4412, 4862, 5806, 6536, 6980])  # Cats population
dogs = np.array([5503, 5222, 5429, 5119, 5175])  # Dogs population
fish = np.array([1530, 1600, 1650, 1700, 1780])  # Fish population
reptiles = np.array([520, 540, 560, 580, 600])   # Reptiles population
rodents = np.array([1080, 1120, 1150, 1200, 1250])  # Rodents population
birds = np.array([880, 900, 930, 960, 980])      # Birds population

# Expenses on pet food and medical care over the years
food_expenses = np.array([551.74, 576.5, 661.52, 715.44, 773.74])  # Food expenses (in billions)
medical_expenses = np.array([483, 655, 881, 1216, 1726])           # Medical expenses (in billions)
```

```python

# A dictionary to store pet data for various species
pet_data = {
    "Cats": cats,
    "Dogs": dogs,
    "Fish": fish,
    "Reptiles": reptiles,
    "Rodents": rodents,
    "Birds": birds
}

# Double Exponential Smoothing function for forecasting
def double_exponential_smoothing(data, alpha, beta, future_periods):
    """Double Exponential Smoothing Model"""
    S = [data[0]]  # Initialize the smoothed series
    b = [data[1] - data[0]]  # Initialize the trend series

    # Apply the Double Exponential Smoothing
    for t in range(1, len(data)):
        S_t = alpha * data[t] + (1 - alpha) * (S[-1] + b[-1])  # Smoothed value
        b_t = beta * (S_t - S[-1]) + (1 - beta) * b[-1]  # Trend value
        S.append(S_t)
        b.append(b_t)

    # Forecast future values based on the trend
    forecast = [S[-1] + (i + 1) * b[-1] for i in range(future_periods)]
    return S, forecast

# Linear Regression function for forecasting (used for food and medical expenses)
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline

def linear_regression_forecast(data, years, future_years):
    """Perform Linear Regression to forecast future data"""
    model = LinearRegression()
    X = np.array(years).reshape(-1, 1)  # Years as independent variable
    y = np.array(data).reshape(-1, 1)  # Data to be predicted (dependent variable)

    # Fit the linear regression model
    model.fit(X, y)

    # Forecast future values
    forecast_years = np.array(future_years).reshape(-1, 1)
```

```
    forecast = model.predict(forecast_years).flatten()

    # Predicted (fitted) values for the training set
    fitted = model.predict(X).flatten()

    # Calculate Mean Squared Error (MSE)
    mse = np.mean((data - fitted) ** 2)

    # Calculate R² (coefficient of determination)
    r2 = model.score(X, y)

    return model.coef_[0, 0], model.intercept_[0], forecast, fitted, mse, r2

def quadratic_regression_forecast(data, years, future_years):
    """Perform Quadratic Regression (degree 2) for forecasting"""
    model = make_pipeline(PolynomialFeatures(degree=2), LinearRegression())
    X = np.array(years).reshape(-1, 1)  # Years as independent variable
    y = np.array(data).reshape(-1, 1)  # Data to be predicted (dependent variable)

    # Fit the quadratic regression model
    model.fit(X, y)

    # Forecast future values
    forecast_years = np.array(future_years).reshape(-1, 1)
    forecast = model.predict(forecast_years).flatten()

    # Predicted (fitted) values for the training set
    fitted = model.predict(X).flatten()

    # Calculate Mean Squared Error (MSE)
    mse = np.mean((data - fitted) ** 2)

    # Calculate R² (coefficient of determination)
    r2 = model.score(X, y)

    return model, forecast, fitted, mse, r2
# Define future years and the number of future periods to forecast
future_years = np.array([2024, 2025, 2026])
future_periods = len(future_years)
alpha, beta = 0.8, 0.2  # Smoothing parameters for double exponential smoothing

# Dictionary to store forecast results for pet populations
pet_forecast_results = {}
for pet, data in pet_data.items():
```

```
    # Apply double exponential smoothing to the pet population data
    smoothed, forecast = double_exponential_smoothing(data, alpha, beta, future_periods)
    # Calculate Mean Squared Error (MSE) for the smoothing process
    mse = np.mean((data - smoothed[:len(data)]) ** 2)
    pet_forecast_results[pet] = {
        "Smoothed": smoothed,
        "Forecast": forecast,
        "MSE": mse
    }

# Forecast food expenses using linear regression
food_coef, food_intercept, food_forecast, food_fit, food_mse, food_r2 = linear_regression_forecast(food_expenses, years, future_years)

# Forecast medical expenses using quadratic regression
medical_model, medical_forecast, medical_fit, medical_mse, medical_r2 = quadratic_regression_forecast(medical_expenses, years, future_years)

# Visualization of the pet population forecast
plt.figure(figsize=(14, 10))
for i, (pet, data) in enumerate(pet_data.items()):
    plt.subplot(2, 3, i + 1)
    plt.plot(years, data, 'o-', label='Historical', markersize=8)  # Plot historical data
    plt.plot(years, pet_forecast_results[pet]["Smoothed"], 'r-', label='Smoothed')  # Smoothed data
    plt.plot(future_years, pet_forecast_results[pet]["Forecast"], 'b--', label='Forecast')  # Forecasted data
    plt.title(f"{pet} Population", fontsize=14)
    plt.xlabel("Year", fontsize=10)
    plt.ylabel("Count (ten-thousands)", fontsize=10)
    plt.legend(fontsize=8)
    plt.grid(alpha=0.3)
plt.tight_layout()
plt.suptitle("Pet Population Forecast (2019-2026)", fontsize=16, y=1.02)
plt.show()

# Visualization of food and medical expenses forecast
plt.figure(figsize=(12, 6))
plt.plot(years, food_expenses, 'o-', label='Food Expenses Historical', markersize=8)  # Historical food expenses
plt.plot(years, food_fit, 'r-', label='Food Expenses Fitted')  # Fitted line for food expenses
plt.plot(future_years, food_forecast, 'b--', label='Food Expenses Forecast')  # Forecasted food expenses

```

```
plt.plot(years, medical_expenses, 'o-', label='Medical Expenses Historical', markersize=8)  # Historical medical expenses
plt.plot(years, medical_fit, 'g-', label='Medical Expenses Fitted')  # Fitted line for medical expenses
plt.plot(future_years, medical_forecast, 'm--', label='Medical Expenses Forecast')  # Forecasted medical expenses

plt.title("Food and Medical Expenses Forecast (2019-2026)", fontsize=16)
plt.xlabel("Year", fontsize=12)
plt.ylabel("Expenses (ten-thousands)", fontsize=12)
plt.legend(fontsize=10)
plt.grid(alpha=0.3)
plt.show()

# Output the results for pet population forecasts
print("Pet Population Forecast Results (Double Exponential Smoothing):")
for pet, result in pet_forecast_results.items():
    print(f"{pet}:")
    print(f"  Smoothed Values: {result['Smoothed']}")
    print(f"  Forecast (2024-2026): {result['Forecast']}")
    print(f"  MSE: {result['MSE']:.2f}")
    print()

# Output the results for food and medical expenses forecasts
print("Food and Medical Expenses Forecast Results:")
print("Food Expenses (Linear Regression):")
print(f"  Coefficient: {food_coef:.2f}")
print(f"  Intercept: {food_intercept:.2f}")
print(f"  Forecast (2024-2026): {food_forecast}")
print(f"  MSE: {food_mse:.2f}")
print(f"  R²: {food_r2:.4f}")
print()

print("Medical Expenses (Quadratic Regression):")
print(f"  Forecast (2024-2026): {medical_forecast}")
print(f"  MSE: {medical_mse:.2f}")
print(f"  R²: {medical_r2:.4f}")

#-----------------------------------------------------------------------code-----------------------------------------#
```

The code for the second question

```python
# Import necessary libraries
#---------p2.py------------
from sklearn.linear_model import LinearRegression
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Create a dictionary with time series data (year and pet demand)
data = {
    'year': [2019, 2020, 2021, 2022, 2023],
    'cat_demand': [9420, 6500, 9420, 7380, 7380],
    'dog_demand': [8970, 8500, 8970, 8970, 8010],
}

# Convert the dictionary into a DataFrame for easy handling and manipulation
df = pd.DataFrame(data)

# Prepare the feature and target variables
X = df['year'].values.reshape(-1, 1)  # Feature variable: year (reshaped for scikit-learn)
y_cat = df['cat_demand'].values  # Target variable for cat demand
y_dog = df['dog_demand'].values  # Target variable for dog demand

# Create linear regression models for both cat and dog demand predictions
model_cat = LinearRegression()
model_dog = LinearRegression()

# Fit the models to the data (learn the relationship between year and demand)
model_cat.fit(X, y_cat)
model_dog.fit(X, y_dog)

# Predict the future demand for the next 3 years (2024, 2025, 2026)
years_future = np.array([2024, 2025, 2026]).reshape(-1, 1)
cat_forecast = model_cat.predict(years_future)  # Predict future cat demand
dog_forecast = model_dog.predict(years_future)  # Predict future dog demand

# Create a DataFrame to display the forecast results for the next 3 years
forecast_df = pd.DataFrame({
    'year': [2024, 2025, 2026],
    'cat_demand_forecast': cat_forecast,
    'dog_demand_forecast': dog_forecast
})

```

```
# Print the forecasted data for cat and dog demand
print(forecast_df)

# Plot the historical and forecasted demand data for cats and dogs
plt.plot(df['year'], df['cat_demand'], label='Actual Cat Demand')  # Historical cat demand
plt.plot(df['year'], df['dog_demand'], label='Actual Dog Demand')  # Historical dog demand
plt.plot([2024, 2025, 2026], cat_forecast, label='Forecasted Cat Demand', linestyle='--')  # Forecasted cat demand
plt.plot([2024, 2025, 2026], dog_forecast, label='Forecasted Dog Demand', linestyle='--')  # Forecasted dog demand
plt.xlabel('Year')  # Label for the x-axis (year)
plt.ylabel('Demand')  # Label for the y-axis (demand)
plt.legend()  # Show legend for the plot
plt.show()  # Display the plot

#-----------------------------------------------------------------------code----------------------------------------#

import numpy as np
import matplotlib.pyplot as plt

# Define the exponential growth function
def exponential_growth(initial_value, growth_rate, years):
    """
    Calculate exponential growth forecast.
    initial_value: initial value (e.g., market size in 2019)
    growth_rate: annual growth rate (e.g., 9.7% or 0.097)
    years: number of years to forecast
    """
    return initial_value * (1 + growth_rate) ** np.arange(1, years + 1)

# U.S. dog food market data (2019 to 2026)
initial_dog_food_2019 = 246  # Dog food market size in 2019 (in million USD)
growth_rate_dog = 0.097  # Annual growth rate for dog food (9.7%)

# U.S. cat food market data (2019 to 2026)
initial_cat_food_2019 = 98  # Cat food market size in 2019 (in million USD)
growth_rate_cat = 0.124  # Annual growth rate for cat food (12.4%)

# Number of years for forecasting (from 2019 to 2026)
years = 8  # From 2019 to 2026, 8 years in total

# Calculate forecasted market sizes from 2019 to 2026
dog_food_forecast = exponential_growth(initial_dog_food_2019, growth_rate_dog, years)
```

```
cat_food_forecast = exponential_growth(initial_cat_food_2019, growth_rate_cat, years)

# Print forecast results
years_range = np.arange(2019, 2027)

print("Forecasted dog food market demand (in million USD):")
for i, year in enumerate(years_range):
    print(f"Year {year}: {dog_food_forecast[i]:.2f} million USD")

print("\nForecasted cat food market demand (in million USD):")
for i, year in enumerate(years_range):
    print(f"Year {year}: {cat_food_forecast[i]:.2f} million USD")

# Plot the forecast
plt.figure(figsize=(10, 6))

# Plot dog food demand
plt.plot(years_range, dog_food_forecast, label='Dog Food Demand', marker='o', color='b')

# Plot cat food demand
plt.plot(years_range, cat_food_forecast, label='Cat Food Demand', marker='o', color='g')

plt.title("Pet Food Market Demand Forecast (2019-2026)")
plt.xlabel("Year")
plt.ylabel("Market Demand (in million USD)")
plt.legend()
plt.grid(True)
plt.xticks(years_range)  # Ensure all years from 2019 to 2026 are shown
plt.tight_layout()

# Show the plot
plt.show()

#-----------------------------------------------------------------------code-----------------------------------------#

import numpy as np
import matplotlib.pyplot as plt

# Initial global pet food market size (2020 data)
initial_market_size = 980.7  # Global pet food market size (in billion USD)
growth_rate_dog = 0.097  # Annual growth rate of dog food market (9.7%)
growth_rate_cat = 0.124  # Annual growth rate of cat food market (12.4%)

```

```python
# Forecast the market demand from 2021 to 2026
years = 6  # Forecast for 6 years (2021-2026)

# Forecast the global dog food market demand for the next 6 years
dog_food_forecast = initial_market_size * (1 + growth_rate_dog) ** np.arange(1, years + 1)

# Forecast the global cat food market demand for the next 6 years
cat_food_forecast = initial_market_size * (1 + growth_rate_cat) ** np.arange(1, years + 1)

# Output the forecast results
years_range = np.arange(2021, 2027)

print("Forecasted global dog food market demand (in billion USD):")
for i, year in enumerate(years_range):
    print(f"Year {year}: {dog_food_forecast[i]:.2f} billion USD")

print("\nForecasted global cat food market demand (in billion USD):")
for i, year in enumerate(years_range):
    print(f"Year {year}: {cat_food_forecast[i]:.2f} billion USD")

# Calculate the variance
dog_food_variance = np.var(dog_food_forecast)
cat_food_variance = np.var(cat_food_forecast)

# Calculate the slope of the trend line
dog_food_slope = np.polyfit(years_range, dog_food_forecast, 1)[0]
cat_food_slope = np.polyfit(years_range, cat_food_forecast, 1)[0]

print(f"\nDog food market demand variance: {dog_food_variance:.2f} billion USD^2")
print(f"Cat food market demand variance: {cat_food_variance:.2f} billion USD^2")
print(f"\nDog food market demand slope: {dog_food_slope:.2f} billion USD per year")
print(f"Cat food market demand slope: {cat_food_slope:.2f} billion USD per year")

# Plot the forecasted market demand for the next 6 years
plt.figure(figsize=(10, 6))

# Plot the dog food market forecast (solid line for 2021-2023, dashed line for 2024-2026)
plt.plot(years_range[:3], dog_food_forecast[:3], label='Dog Food Demand (2021-2023)', marker='o', color='b', linestyle='-', linewidth=2)
plt.plot(years_range[3:], dog_food_forecast[3:], marker='o', color='b', linestyle='--', linewidth=2)

# Plot the cat food market forecast (solid line for 2021-2023, dashed line for 2024-2026)
```

```
plt.plot(years_range[:3], cat_food_forecast[:3], label='Cat Food Demand (2021-2023)', marker='o', color='g', linestyle='-', linewidth=2)
plt.plot(years_range[3:], cat_food_forecast[3:], marker='o', color='g', linestyle='--', linewidth=2)

# Plot dotted lines connecting 2023 and 2024 for both dog and cat food markets
plt.plot(years_range[2:4], dog_food_forecast[2:4], color='b', linestyle=':', linewidth=2)
plt.plot(years_range[2:4], cat_food_forecast[2:4], color='g', linestyle=':', linewidth=2)

# Add title and labels
plt.title("Global Pet Food Market Demand Forecast (2021-2026)")
plt.xlabel("Year")
plt.ylabel("Market Demand (in billion USD)")
plt.legend()
plt.grid(True)
plt.xticks(years_range)  # Show the years from 2021 to 2026
plt.tight_layout()

# Display the chart
plt.show()

#-----------------------------------------------------------------------code-----------------------------------------#
import numpy as np
import matplotlib.pyplot as plt

# Define the coefficients of the regression equation
coefficients = [9418.77, -11.24, 5.47, -0.14, 0.02, -0.01]

# Define the regression equation
def regression_eq(x, coefficients):
    y = coefficients[0]
    for i in range(1, len(coefficients)):
        y += coefficients[i] * (x ** i)
    return y

# Generate years from 2019 to 2026
years = np.arange(2019, 2027)

# Calculate predicted values for each year
predicted_values = [regression_eq(year, coefficients) for year in years]

# Create the plot
plt.figure(figsize=(10, 6))
```

```python

# Plot the regression curve
plt.plot(years, predicted_values, 'r-', label="Regression Curve", marker='o')

# Set the title and labels
plt.title("US forecast_cat (2019-2026)", fontsize=14)
plt.xlabel("Year", fontsize=12)
plt.ylabel("Predicted Value", fontsize=12)

# Add grid for better visibility
plt.grid(True)

# Display the legend
plt.legend()

# Show the plot
plt.show()

#-----------------------------------------------------------------------code-----------------------------------------#

import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression
import matplotlib.pyplot as plt
import scipy.stats as stats

# 1. Data preparation: Organize the data into a dictionary
data = {
    'year': [2019, 2020, 2021, 2022, 2023],  # Year data
    'US_cat': [9420, 6500, 9420, 9420, 7380],  # US cat demand data
    'US_dog': [8970, 8500, 8970, 8970, 8010],  # US dog demand data
    'France_cat': [1300, 1490, 1510, 1510, 1490],  # France cat demand data
    'France_dog': [740, 775, 750, 750, 760],  # France dog demand data
    'Germany_cat': [1470, 1570, 1670, 1670, 1520],  # Germany cat demand data
    'Germany_dog': [1010, 1070, 1030, 1030, 1060],  # Germany dog demand data
    'China_cat': [4412, 4862, 5806, 6536, 6980],  # China cat demand data
    'China_dog': [5503, 5222, 5429, 5119, 5175],  # China dog demand data
}

# Convert the data dictionary to a DataFrame for easier manipulation
df = pd.DataFrame(data)

# 2. Train models: Predict the number of cats and dogs for future years (2024-2026)
```

```
X = df['year'].values.reshape(-1, 1)  # Independent variable: year data
forecast_years = np.array([2024, 2025, 2026]).reshape(-1, 1)  # Years to forecast

# Dictionary to store the forecast results for each country
forecast_results = {}

# Perform linear regression for each country's cat and dog demand
for country in ['US', 'France', 'Germany', 'China']:
    # Get the actual cat and dog demand data for the current country
    y_cat = df[f'{country}_cat'].values
    y_dog = df[f'{country}_dog'].values

    # Initialize linear regression models for cats and dogs
    model_cat = LinearRegression()
    model_dog = LinearRegression()

    # Fit the models to the existing data
    model_cat.fit(X, y_cat)
    model_dog.fit(X, y_dog)

    # Predict the cat and dog demand for the forecast years
    forecast_cat = model_cat.predict(forecast_years)
    forecast_dog = model_dog.predict(forecast_years)

    # Store the forecast results for the current country
    forecast_results[country] = {
        'forecast_cat': forecast_cat,
        'forecast_dog': forecast_dog,
    }

# 3. Plot the trends for cat and dog demand across different countries
plt.figure(figsize=(12, 8))  # Set the size of the overall figure

# Plot data for each country in a subplot
for i, country in enumerate(['US', 'France', 'Germany', 'China'], 1):
    plt.subplot(2, 2, i)  # Create a 2x2 grid of subplots

    # Plot actual cat and dog data as lines with markers
    plt.plot(df['year'], df[f'{country}_cat'], label='Cat Actual', marker='o')
    plt.plot(df['year'], df[f'{country}_dog'], label='Dog Actual', marker='o')

    # Plot the forecasted cat and dog data for future years (2024-2026)
    plt.plot([2024, 2025, 2026], forecast_results[country]['forecast_cat'], label='Cat Forecast',
linestyle='--')
```

```
    plt.plot([2024, 2025, 2026], forecast_results[country]['forecast_dog'], label='Dog Forecast', linestyle='--')

    # Set the title and labels for the current subplot
    plt.title(f"{country} - Cat & Dog Demand")
    plt.xlabel("Year")
    plt.ylabel("Demand")
    plt.legend()  # Show the legend

# Adjust the layout to prevent overlap
plt.tight_layout()

# Display the plot
plt.show()

#-----------------------------------------------------------------------code-----------------------------------------#

import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression

# Data for the number of cats and dogs in Germany
years = np.array([2019, 2020, 2021, 2022, 2023])
cats = np.array([1470, 1570, 1670, 1670, 1520])  # Number of cats in Germany
dogs = np.array([1010, 1070, 1030, 1030, 1060])  # Number of dogs in Germany

# 1. Perform quadratic regression fitting for the cat data
# Use PolynomialFeatures to generate quadratic features
poly_cat = PolynomialFeatures(degree=2)
years_cat_poly = poly_cat.fit_transform(years.reshape(-1, 1))

# Create a regression model and fit it
reg_cat = LinearRegression()
reg_cat.fit(years_cat_poly, cats)

# Predict the number of cats for the years 2024, 2025, and 2026
future_years = np.array([2024, 2025, 2026]).reshape(-1, 1)
future_years_cat_poly = poly_cat.transform(future_years)
cats_pred = reg_cat.predict(future_years_cat_poly)

# 2. Perform cubic regression fitting for the dog data
# Use PolynomialFeatures to generate cubic features
```

```
poly_dog = PolynomialFeatures(degree=3)
years_dog_poly = poly_dog.fit_transform(years.reshape(-1, 1))

# Create a regression model and fit it
reg_dog = LinearRegression()
reg_dog.fit(years_dog_poly, dogs)

# Predict the number of dogs for the years 2024, 2025, and 2026
future_years_dog_poly = poly_dog.transform(future_years)
dogs_pred = reg_dog.predict(future_years_dog_poly)

# 3. Calculate the variance (using historical data and predicted data to calculate variance)
# Calculate the variance for the cats prediction using the last 3 years of actual data
cat_variance = np.var(cats_pred - cats[-3:])  # Use the last 3 years of data for the variance calculation
# Calculate the variance for the dogs prediction using the last 3 years of actual data
dog_variance = np.var(dogs_pred - dogs[-3:])

# 4. Plot the regression curves
plt.figure(figsize=(12, 6))

# Plot the regression curve for the number of cats
plt.subplot(1, 2, 1)
plt.scatter(years, cats, color='blue', label='Actual Cats Data')
plt.plot(years, reg_cat.predict(years_cat_poly), color='red', label='Fitted Regression Curve')
plt.plot(future_years, cats_pred, color='green', linestyle='--', label='Predicted Cats (2024-2026)')
plt.title('Germany Cat Number Prediction')
plt.xlabel('Year')
plt.ylabel('Number of Cats')
plt.legend()

# Plot the regression curve for the number of dogs
plt.subplot(1, 2, 2)
plt.scatter(years, dogs, color='blue', label='Actual Dogs Data')
plt.plot(years, reg_dog.predict(years_dog_poly), color='red', label='Fitted Regression Curve')
plt.plot(future_years, dogs_pred, color='green', linestyle='--', label='Predicted Dogs (2024-2026)')
plt.title('Germany Dog Number Prediction')
plt.xlabel('Year')
plt.ylabel('Number of Dogs')
plt.legend()

# Show the plots
```

```
plt.tight_layout()
plt.show()

# Output the variance for the predictions
print(f"Variance for Cats Prediction: {cat_variance}")
print(f"Variance for Dogs Prediction: {dog_variance}")

#----------------------------------------------------------------------code----------------------
---------------------------------------#
```

The code for the Third question

The first model

```python
#-----------p3_1.py-----------------
import pandas as pd
import numpy as np
from statsmodels.tsa.holtwinters import ExponentialSmoothing
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import matplotlib.dates as mdates

# Set the font to support Chinese characters in plots
matplotlib.rcParams['font.family'] = 'SimHei'
matplotlib.rcParams['axes.unicode_minus'] = False

# Create hypothetical data: Historical production (in 10,000 tons) and export value (in USD billion) for pet food in China
data = {
    'year': pd.date_range(start='2019-01-01', periods=5, freq='YE'),
    'production': [440.7, 727.3, 1554, 1508, 2793],  # Production data
    'export': [21.3, 9.8, 12.2, 24.7, 39.6],        # Export value data
    'market size': [1136.37, 1236.37, 1349.42, 1409, 1339]  # Market size data
}

# Create a DataFrame from the data
df = pd.DataFrame(data)
df.set_index('year', inplace=True)  # Set 'year' as the index
df.index.freq = 'YE'  # Set the frequency to year-end ('YE')

# Forecast for production values
train_production = df['production']
model_production = ExponentialSmoothing(train_production, trend='add', seasonal=None)  # Use additive trend, no seasonal component
fit_production = model_production.fit()  # Fit the model

# Extract the smoothing parameters for the production model
alpha_production = fit_production.params['smoothing_level']  # Level smoothing parameter
beta_production = fit_production.params.get('smoothing_slope') if 'smoothing_slope' in fit_production.params else None  # Trend smoothing parameter

# Forecast the next 3 years of production
production_forecast = fit_production.forecast(steps=3)
```

```python

# Create a DataFrame for the forecasted production values
forecast_years = pd.date_range(start='2024-01-01', periods=3, freq='YE')
df_forecast_production = pd.DataFrame({'predicted production': production_forecast}, index=forecast_years)

# Forecast for export values
train_export = df['export']
model_export = ExponentialSmoothing(train_export, trend='add', seasonal=None)  # Use additive trend, no seasonal component
fit_export = model_export.fit()  # Fit the model

# Extract the smoothing parameters for the export model
alpha_export = fit_export.params['smoothing_level']  # Level smoothing parameter
beta_export = fit_export.params.get('smoothing_slope') if 'smoothing_slope' in fit_export.params else None  # Trend smoothing parameter

# Forecast the next 3 years of export values
export_forecast = fit_export.forecast(steps=3)

# Create a DataFrame for the forecasted export values
df_forecast_export = pd.DataFrame({'predicted export value': export_forecast}, index=forecast_years)

# Forecast for market size
train_market = df['market size']
model_market = ExponentialSmoothing(train_market, trend='add', seasonal=None)  # Use additive trend, no seasonal component
fit_market = model_market.fit()  # Fit the model

# Extract the smoothing parameters for the market size model
alpha_market = fit_market.params['smoothing_level']  # Level smoothing parameter
beta_market = fit_market.params.get('smoothing_slope') if 'smoothing_slope' in fit_market.params else None  # Trend smoothing parameter

# Forecast the next 3 years of market size
market_forecast = fit_market.forecast(steps=3)

# Create a DataFrame for the forecasted market size values
df_forecast_market = pd.DataFrame({'predicted market size': market_forecast}, index=forecast_years)

# Combine the original data with the forecasted data for production, export, and market size
```

```
df_combined = pd.concat([df, df_forecast_production, df_forecast_export, df_forecast_market], axis=1)

# Format the index as strings representing just the year
df_combined.index = df_combined.index.year.astype(str)

# The df_combined DataFrame now contains both historical and forecasted data
# Output the combined DataFrame with both historical and forecasted data
print(df_combined.to_string(index=True))

# Construct expressions for the smoothing parameters and trends for each variable
production_expression = f"Production: Level = {alpha_production:.4f}, Trend = {beta_production:.4f}" if beta_production is not None else f"Production: Level = {alpha_production:.4f}, Trend = N/A"
export_expression = f"Export: Level = {alpha_export:.4f}, Trend = {beta_export:.4f}" if beta_export is not None else f"Export: Level = {alpha_export:.4f}, Trend = N/A"
market_expression = f"Market Size: Level = {alpha_market:.4f}, Trend = {beta_market:.4f}" if beta_market is not None else f"Market Size: Level = {alpha_market:.4f}, Trend = N/A"

# Output the smoothing parameters (level and trend) for each variable
print(production_expression)
print(export_expression)
print(market_expression)

# Construct and output the specific exponential smoothing formula for forecasting each variable
production_formula = f"Production Forecast: Y_t = {alpha_production:.4f} + {beta_production:.4f} * (t - 1)" if beta_production is not None else f"Production Forecast: Y_t = {alpha_production:.4f}"
export_formula = f"Export Forecast: Y_t = {alpha_export:.4f} + {beta_export:.4f} * (t - 1)" if beta_export is not None else f"Export Forecast: Y_t = {alpha_export:.4f}"
market_formula = f"Market Size Forecast: Y_t = {alpha_market:.4f} + {beta_market:.4f} * (t - 1)" if beta_market is not None else f"Market Size Forecast: Y_t = {alpha_market:.4f}"

# Output the forecasting formulas
print(production_formula)
print(export_formula)
print(market_formula)

# Visualize the historical data and forecasted values
plt.figure(figsize=(12, 6))  # Set the figure size

# Plot the historical production, export, and market size data
plt.plot(df.index, df['production'], marker='o', label='historical production')  # Historical production data
plt.plot(df.index, df['export'], marker='o', label='historical export value')  # Historical export data
```

```
plt.plot(df.index, df['market size'], marker='o', label='historical market size')  # Historical market size data

# Plot the forecasted production, export, and market size values
plt.plot(df_forecast_production.index, df_forecast_production['predicted production'], marker='o', linestyle='--', color='orange', label='predicted production')  # Predicted production
plt.plot(df_forecast_export.index, df_forecast_export['predicted export value'], marker='o', linestyle='--', color='green', label='predicted export value')  # Predicted export value
plt.plot(df_forecast_market.index, df_forecast_market['predicted market size'], marker='o', linestyle='--', color='red', label='predicted market size')  # Predicted market size

# Format the x-axis to display years
plt.gca().xaxis.set_major_formatter(mdates.DateFormatter('%Y'))  # Set x-axis labels as years
plt.gca().xaxis.set_major_locator(mdates.YearLocator(1))  # Set major ticks at yearly intervals
plt.xlim(df.index.min(), df_forecast_export.index.max())  # Set the x-axis limits from the first year to the last forecast year
plt.xticks(df.index.append(df_forecast_production.index), rotation=45)  # Rotate x-axis labels for readability

# Add title and axis labels
plt.title('Actual and forecasted production, export value and market size of pet food in China')  # Set plot title
plt.xlabel('year')  # Set x-axis label
plt.ylabel('value')  # Set y-axis label
plt.legend()  # Display the legend
plt.grid()  # Display grid lines

# Save the plot as a PNG file
plt.savefig('pet_food_forecast.png')  # Save the plot to a file named 'pet_food_forecast.png'

#-----------------------------------------------------------------------code----------------------------------------#
```

The second model

```
#--------------p3_2.py---------------------
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use('Agg')  # Use Agg backend to avoid graphical interface issues
import matplotlib.pyplot as plt
from matplotlib import font_manager
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import RidgeCV  # Use Ridge regression
```

```python
from sklearn.model_selection import KFold, cross_val_score

# Set Chinese font
font_path = 'C:/Windows/Fonts/msyh.ttc'  # Path to font file
font_prop = font_manager.FontProperties(fname=font_path)  # Create font properties
plt.rcParams['font.family'] = font_prop.get_name()  # Set global font
matplotlib.rcParams['axes.unicode_minus'] = False  # Fix negative sign display issues

# Prepare data
data = {
    'Year': [2019, 2020, 2021, 2022, 2023],  # Year data
    'Production': [440.7, 727.3, 1554, 1508, 2793],  # Production volume data
    'Exports': [21.3, 9.8, 12.2, 24.7, 39.6],  # Export volume data
    'market': [1136.7, 1236.7, 1349.42, 1409, 1339]  # Market size data
}

df = pd.DataFrame(data)  # Create DataFrame
df['Relative_Year'] = df['Year'] - df['Year'].min()  # Calculate relative years

# Select independent and dependent variables
X = df[['Relative_Year']]  # Independent variable: relative years
y_production = df['Production']  # Dependent variable: production volume
y_exports = df['Exports']  # Dependent variable: export volume
y_market = df['market']  # Dependent variable: market size

# Create a k-fold cross-validation object
kf = KFold(n_splits=2)  # Use 2-fold cross-validation

# Initialize the best model's order and score
best_degree_production = 0
best_score_production = float('inf')  # Initialize to positive infinity (MSE)

best_degree_exports = 0
best_score_exports = float('inf')

best_degree_market = 0
best_score_market = float('inf')

# Try different polynomial orders
for degree in range(1, 6):  # Try orders from 1 to 5
    # Create polynomial features
    poly_features = PolynomialFeatures(degree=degree)
    X_poly = poly_features.fit_transform(X)  # Transform independent variable into polynomial features
```

```python

        # Compute cross-validation scores (use negative mean squared error)
        cv_production = cross_val_score(RidgeCV(alphas=[0.1, 1.0, 10.0], store_cv_results=True), X_poly, y_production, cv=kf, scoring='neg_mean_squared_error')
        cv_exports = cross_val_score(RidgeCV(alphas=[0.1, 1.0, 10.0], store_cv_results=True), X_poly, y_exports, cv=kf, scoring='neg_mean_squared_error')
        cv_market = cross_val_score(RidgeCV(alphas=[0.1, 1.0, 10.0], store_cv_results=True), X_poly, y_market, cv=kf, scoring='neg_mean_squared_error')

        # Calculate the mean score (take the negative value as we use negative MSE)
        mean_production_score = -np.mean(cv_production)
        mean_exports_score = -np.mean(cv_exports)
        mean_market_score = -np.mean(cv_market)

        # Update the best model information
        if mean_production_score < best_score_production:  # Minimize MSE
            best_score_production = mean_production_score
            best_degree_production = degree

        if mean_exports_score < best_score_exports:
            best_score_exports = mean_exports_score
            best_degree_exports = degree

        if mean_market_score < best_score_market:
            best_score_market = mean_market_score
            best_degree_market = degree

    # Output the best order and corresponding score
    print(f"Best Degree for Production: {best_degree_production} with Score: {best_score_production:.2f}")
    print(f"Best Degree for Exports: {best_degree_exports} with Score: {best_score_exports:.2f}")
    print(f"Best Degree for Market: {best_degree_market} with Score: {best_score_market:.2f}")

    # Fit models using the best order
    best_poly_features_production = PolynomialFeatures(degree=best_degree_production)
    X_poly_production = best_poly_features_production.fit_transform(X)  # Transform into polynomial features
    model_production = RidgeCV(alphas=[0.1, 1.0, 10.0]).fit(X_poly_production, y_production)  # Fit model

    best_poly_features_exports = PolynomialFeatures(degree=best_degree_exports)
    X_poly_exports = best_poly_features_exports.fit_transform(X)  # Transform into polynomial features
    model_exports = RidgeCV(alphas=[0.1, 1.0, 10.0]).fit(X_poly_exports, y_exports)  # Fit model

```

```
best_poly_features_market = PolynomialFeatures(degree=best_degree_market)
X_poly_market = best_poly_features_market.fit_transform(X)  # Transform into polynomial features
model_market = RidgeCV(alphas=[0.1, 1.0, 10.0]).fit(X_poly_market, y_market)  # Fit model

# Output model equations
def print_model_equation(model):
    coef = model.coef_  # Get model coefficients
    intercept = model.intercept_  # Get intercept
    equation = f"y = {intercept:.2f}"  # Build equation string
    for i in range(1, len(coef)):
        equation += f" + ({coef[i]:.2f}) * x^{i}"  # Add each term
    return equation

# Print model equations
print("Production Model Equation:", print_model_equation(model_production))
print("Exports Model Equation:", print_model_equation(model_exports))
print("Market Model Equation:", print_model_equation(model_market))

# Predict future production and exports
future_years = pd.DataFrame({'Relative_Year': [5, 6, 7]})  # Relative years for the next 3 years
future_years_poly_production = best_poly_features_production.transform(future_years)  # Transform into polynomial features
future_years_poly_exports = best_poly_features_exports.transform(future_years)  # Transform into polynomial features
future_years_poly_market = best_poly_features_market.transform(future_years)  # Transform into polynomial features

# Make predictions
predicted_production = model_production.predict(future_years_poly_production)
predicted_exports = model_exports.predict(future_years_poly_exports)
predicted_market = model_market.predict(future_years_poly_market)

# Print prediction results
for i, year in enumerate(future_years['Relative_Year'] + 2019):  # Assume 2024 as the starting point
    print(f"Predicted Production for {year}: {predicted_production[i]:.2f} million tons")
    print(f"Predicted Exports for {year}: {predicted_exports[i]:.2f} billion USD")
    print(f"Predicted Market for {year}: {predicted_market[i]:.2f} billion USD")

# Visualize results
plt.figure(figsize=(12, 6))  # Set figure size

# Production plot
plt.subplot(1, 3, 1)  # Create subplot
```

```
plt.scatter(df['Year'], df['Production'], color='blue', label='Actual Production')  # Actual production scatter plot
plt.plot(df['Year'], model_production.predict(X_poly_production), color='red', label='Predicted Production')  # Fitted curve
plt.scatter([2024, 2025, 2026], predicted_production, color='green', label='Future Predictions')  # Future predictions
plt.title('The Production of Pet Food')  # Title
plt.xlabel('Year')  # X-axis label
plt.ylabel('Production (million tons)')  # Y-axis label
plt.legend()  # Show legend

# Export plot
plt.subplot(1, 3, 2)  # Create subplot
plt.scatter(df['Year'], df['Exports'], color='orange', label='Actual Exports')  # Actual export scatter plot
plt.plot(df['Year'], model_exports.predict(X_poly_exports), color='red', label='Predicted Exports')  # Fitted curve
plt.scatter([2024, 2025, 2026], predicted_exports, color='green', label='Future Predictions')  # Future predictions
plt.title('The Export Value of Pet Food')  # Title
plt.xlabel('Year')  # X-axis label
plt.ylabel('Export Value (billion USD)')  # Y-axis label
plt.legend()  # Show legend

# Market size plot
plt.subplot(1, 3, 3)  # Create subplot
plt.scatter(df['Year'], df['market'], color='yellow', label='Actual Market')  # Actual market size scatter plot
plt.plot(df['Year'], model_market.predict(X_poly_market), color='red', label='Predicted Market')  # Fitted curve
plt.scatter([2024, 2025, 2026], predicted_market, color='green', label='Future Predictions')  # Future predictions
plt.title('Global Pet Food Market Size')  # Title
plt.xlabel('Year')  # X-axis label
plt.ylabel('Market Size (billion USD)')  # Y-axis label
plt.legend()  # Show legend

plt.tight_layout()  # Adjust layout
plt.savefig('pet_food_predictions.png')  # Save figure
```

The code for the fourth question

```
#--------------p4.py----------------
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

# Define historical data
# Export values (in billion USD)
export_data = [22.1, 9.8, 12.2, 24.7, 39.6]  # Historical export values
tariff_data = [0.05, 0.061, 0.054, 0.054, 0.054]  # Corresponding tariff rates (%)
demand_growth = [0.3, 0.2, 0.25, 0.3, 0.4]  # Global demand growth rates

# Simplified model fitting
# Calculate year-on-year growth rate (y)
growth_rate = np.array(export_data[1:]) / np.array(export_data[:-1])  # Response variable

# Ensure the independent and dependent variables align
X = np.array([(1 - t) * (1 + d) for t, d in zip(tariff_data[:-1], demand_growth[:-1])])  # Independent variable (using first 4 data points)
y = growth_rate  # Dependent variable

# Log transformation
log_X = np.log(X).reshape(-1, 1)  # Log of the independent variable
log_y = np.log(y)  # Log of the dependent variable

# Linear regression fitting
model = LinearRegression()
model.fit(log_X, log_y)

# Extract the fitted parameters
delta = -model.coef_[0]  # Sensitivity coefficient for tariff
nu = model.intercept_  # Demand promotion effect
print(f"Fitted parameters: delta={delta:.4f}, nu={nu:.4f}")

# Step 3: Predict future exports for the next three years
future_years = [2024, 2025, 2026]  # Years for prediction

# Scenario 1: Constant tariffs
constant_tariffs = [0.057, 0.054, 0.057]  # Tariffs remain constant
future_demand_growth = [0.42, 0.45, 0.5]  # Assumed demand growth rates for the next three years

# Scenario 2: Increasing tariffs
increasing_tariffs = [0.057, 0.063, 0.071]
```

```python

# Scenario 3: Decreasing tariffs
decreasing_tariffs = [0.051, 0.045, 0.029]

# Prediction function
def predict_future_exports(tariffs, demand_growth, initial_export):
    future_exports = []
    current_export = initial_export
    for t, d in zip(tariffs, demand_growth):
        multiplier = (1 - delta * t) * (1 + nu * d)
        future_export = current_export * multiplier
        future_exports.append(future_export)
        current_export = future_export
    return future_exports

# Predict for the three scenarios
future_exports_constant = predict_future_exports(constant_tariffs, future_demand_growth, export_data[-1])
future_exports_increasing = predict_future_exports(increasing_tariffs, future_demand_growth, export_data[-1])
future_exports_decreasing = predict_future_exports(decreasing_tariffs, future_demand_growth, export_data[-1])

# Print predicted results
print("Future export predictions (Constant Tariffs, billion USD):", future_exports_constant)
print("Future export predictions (Increasing Tariffs, billion USD):", future_exports_increasing)
print("Future export predictions (Decreasing Tariffs, billion USD):", future_exports_decreasing)

# Step 4: Visualization
plt.figure(figsize=(10, 6))

# Plot historical export values
years = [2019, 2020, 2021, 2022, 2023]  # Historical years
plt.plot(years, export_data, label="Historical Exports", marker='o')

# Plot the predicted values for the three scenarios
plt.plot(future_years, future_exports_constant, label="Prediction (Constant Tariffs)", marker='o', linestyle="--")
plt.plot(future_years, future_exports_increasing, label="Prediction (Increasing Tariffs)", marker='o', linestyle="--")
plt.plot(future_years, future_exports_decreasing, label="Prediction (Decreasing Tariffs)", marker='o', linestyle="--")

# Set chart title and labels
```

```
plt.xlabel("Year")  # X-axis label
plt.ylabel("Exports (Billion USD)")  # Y-axis label
plt.title("Exports Prediction under Different Tariff Scenarios")  # Chart title
plt.legend()  # Legend

# Show grid lines
plt.grid(alpha=0.3)

plt.tight_layout()
plt.show()

#-----------------------------------------------------------------------code-----------------------------------------#

import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Define data
years = np.array([2019, 2020, 2021, 2022, 2023]).reshape(-1, 1)  # Historical years
exchange_rates = np.array([6.9098, 6.9001, 6.45, 6.7313, 7.0764])  # Historical exchange rates
global_demand_growth = np.array([0.3, 0.2, 0.25, 0.3, 0.4])  # Updated global demand growth rates
export_data = np.array([22.1, 9.8, 12.2, 24.7, 39.6])  # Historical export volume (in billion USD)

# Combine features into a single matrix
features = np.hstack((years, exchange_rates.reshape(-1, 1), global_demand_growth.reshape(-1, 1)))

# Linear regression model
model = LinearRegression()
model.fit(features, export_data)  # Fit the model using multiple features

# Predict fitted values for historical data
fitted_values = model.predict(features)

# Construct future data
future_years = np.array([2024, 2025, 2026]).reshape(-1, 1)
future_exchange_rates = np.array([7.1854, 7.19125, 7.3255]).reshape(-1, 1)  # Future exchange rates
future_demand_growth = np.array([0.42, 0.45, 0.5]).reshape(-1, 1)  # Updated future global demand growth rates

# Construct feature matrix for future data
future_features = np.hstack((future_years, future_exchange_rates, future_demand_growth))
```

```python

# Predict future export volumes
future_predictions = model.predict(future_features)

# Model evaluation
mse = mean_squared_error(export_data, fitted_values)  # Mean squared error
r2 = r2_score(export_data, fitted_values)  # R² (coefficient of determination)

# Output model evaluation
print(f"Model Mean Squared Error (MSE): {mse:.4f}")
print(f"Model R² (Coefficient of Determination): {r2:.4f}")

# Visualization
plt.figure(figsize=(10, 6))

# Plot historical data
plt.plot(years, export_data, color='blue', label='Historical Data', marker='o', linestyle='-')

# Plot future predictions
plt.plot(future_years, future_predictions, color='orange', label='Future Predictions', marker='o', linestyle='-')

# Add title and labels
plt.title("Exports Prediction by Year (Linear Regression with Updated Features)")
plt.xlabel("Year")  # X-axis is the year
plt.ylabel("Export Volume (Billion USD)")  # Y-axis is export volume
plt.legend()

plt.grid(alpha=0.3)

# Check model parameters
print("Model Coefficients:", model.coef_)
print("Model Intercept:", model.intercept_)

# Check historical fitted values
print("Historical Fitted Values:", fitted_values)

# Check future feature matrix
print("Future Feature Matrix:", future_features)

# Output future predictions
print("Export Predictions for 2024-2026 (Linear Regression, in Billion USD):", future_predictions)

# Display the plot
```

```
plt.tight_layout()
plt.show()

#----------------------------------------------------------------------code----------------------
----------------------------------------#
```